A project report on

EVIDENCE-BASED FACT VERIFICATION USING RETRIEVAL-AUGMENTED GENERATION AND LARGE LANGUAGE MODELS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering

by

DIVIK DHIMAN (23BCE1365)
SAFAL GOYAL (23BCE1234)
SATYAM JHA (23BCE1139)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
SCOPE

Vellore Institute of Technology, Chennai

October, 2026

EVIDENCE-BASED FACT VERIFICATION USING RETRIEVAL-AUGMENTED GENERATION AND LARGE LANGUAGE MODELS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering

by

DIVIK DHIMAN (23BCE1365)
SAFAL GOYAL (23BCE1234)
SATYAM JHA (23BCE1139)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
SCOPE

Vellore Institute of Technology, Chennai

October, 2026

VIT

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby declare that the thesis entitled "Evidence-Based Fact Verification using Retrieval-Augmented Generation and Large Language Models", submitted by Safal Goyal (23BCE1234) for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai, is a record of bonafide work carried out by me under the supervision of Dr. Rabindra Kumar Singh.

The existing repository and research publications used in this study are acknowledged in the report. The report distinguishes the reviewed implementation from the analysis, checks, and proposed improvements described here.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: 20/10/26

Signature of the Candidate

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "Evidence-Based Fact Verification using Retrieval-Augmented Generation (RAG) and Large Language Models" is prepared and submitted by Safal Goyal(23BCE1234) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering, and is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr./Prof. RABINDRA KUMAR SINGH

Date: 20/10/26

Signature of Examiner 1:

Name: RAJESH KUMAR

Date: 21/10/26

Signature of Examiner 2:

Name: OMANA J

Date: 21.10.2026

Approved by the Head of Department

Dr. Prassanna J

Computer Science and Engineering

Date: _____

VELLORE INSTITUTE OF TECHNOLOGY
SCHOOL OF COMPUTER SCIENCE & ENGINEERING (SCOPE)
Chennai-600127

2

ABSTRACT

Document question answering requires more than retrieving text that shares a topic with a question. The selected passages must contain the facts needed for an answer, and the system must respond clearly when that evidence is absent. This report examines Adaptive CRAG, a Python application that combines hybrid retrieval with a corrective workflow for document-based conversation.

The implementation uses Cohere embeddings and Pinecone for dense retrieval, BM25 for lexical retrieval, and reciprocal rank fusion to combine the ranked passages. A Gemini-based verifier assesses local evidence. When evidence is insufficient, the system reformulates the query and repeats retrieval within a fixed attempt budget. Tavily provides an optional web-search fallback. LangGraph manages the workflow, while Streamlit and a terminal interface provide user access. PostgreSQL can retain conversation history.

The study analyses the complete application source at a recorded commit and relates its design to twelve research papers. Seventeen isolated offline assertions examine ranking, source formatting, metadata, identifiers, routing, and annotation behaviour. The checks confirm several deterministic contracts and reproduce important limitations: factual requests can match an overly broad conversational rule, web results bypass evidence verification, and an unresolved annotation name fails under Python 3.12. Source inspection also identifies risks in corpus synchronisation and confidence interpretation.

The resulting system is a useful modular prototype whose answer-quality benefits remain to be measured. The report proposes a controlled evaluation against simpler retrieval baselines and prioritises consistent evidence handling, reproducible configuration, document version management, and claim-level citations. It separates implemented behaviour and executed checks from future experiments, avoiding unsupported claims of accuracy or deployment readiness.

Keywords: retrieval-augmented generation, corrective retrieval, hybrid search, evidence verification, query rewriting, document question answering.

ACKNOWLEDGEMENT

I express my sincere thanks to my guide, Dr. Rabindra Kumar Singh, Associate Professor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for guidance during the preparation of this project report.

I thank Dr. Prassanna J, Head of Department, and the faculty and staff of the School of Computer Science and Engineering for the academic support and facilities provided during the programme.

I acknowledge my project teammates, Divik Dhiman and Satyam Kumar Jha, and the shared effort associated with the project. I also acknowledge the maintainers of the CRAG repository and the researchers whose published work forms the foundation of this study. Their contributions are identified in the text and references.

I am grateful to my family and friends for their encouragement and support throughout the course of the work.

Place: Chennai

Date: **Safal Goyal**

CONTENTS

Chapter 1 INTRODUCTION....11
1.1 BACKGROUND....11
1.2 PROBLEM STATEMENT....11
1.3 OBJECTIVES....12
1.4 SCOPE....12
1.5 SIGNIFICANCE OF THE PROJECT....13
1.6 ORGANISATION OF THE REPORT....13
Chapter 2 BACKGROUND AND LITERATURE REVIEW....14
2.1 REVIEW APPROACH....14
2.2 RETRIEVAL AUGMENTED GENERATION....14
2.3 DENSE PASSAGE RETRIEVAL....15
2.4 RETRIEVAL AUGMENTED PRETRAINING....15
2.5 RECIPROCAL RANK FUSION....15
2.6 FUSION IN DECODER....16
2.7 SELF RAG....16
2.8 CORRECTIVE RETRIEVAL AUGMENTED GENERATION....17
2.9 ADAPTIVE RETRIEVAL....17
2.10 QUERY REWRITING....18
2.11 CONTEXT POSITION AND LENGTH....18
2.12 AUTOMATED RAG EVALUATION....18
2.13 CITATION QUALITY....19
2.14 SYNTHESIS AND RESEARCH GAP....19
Chapter 3 SYSTEM ANALYSIS AND REQUIREMENTS....21
3.1 EXISTING APPROACHES....21
3.2 FUNCTIONAL REQUIREMENTS....21
3.3 QUALITY REQUIREMENTS....22
3.4 DATA AND CORPUS REQUIREMENTS....23
3.5 FEASIBILITY....24
3.6 ASSUMPTIONS AND CONSTRAINTS....24
Chapter 4 SYSTEM DESIGN....26
4.1 ARCHITECTURAL OVERVIEW....26
4.2 INGESTION DESIGN....27
4.3 QUERY STATE....28

4.4 CONTROL FLOW 28
4.5 HYBRID RETRIEVAL DESIGN 30
4.6 EVIDENCE AND RESPONSE CONTRACTS 30
4.7 DATABASE DESIGN 31
4.8 INTERFACE DESIGN 32
4.9 DESIGN ALTERNATIVES 32
Chapter 5 IMPLEMENTATION 33
5.1 ENTRY POINTS AND INITIALISATION 33
5.2 CONFIGURATION 33
5.3 DOCUMENT LOADERS 34
5.4 CHUNKING AND INGESTION 35
5.5 VECTOR STORE WRAPPER 35
5.6 KEYWORD RETRIEVAL AND FUSION 36
5.7 QUERY ROUTING AND REWRITING 36
5.8 VERIFICATION AND GENERATION 37
5.9 WEB FALLBACK 38
5.10 CONVERSATION STORAGE 38
5.11 IMPLEMENTATION BOUNDARIES 39
Chapter 6 TESTING AND EVALUATION METHODOLOGY 40
6.1 EVIDENCE LEVELS 40
6.2 OFFLINE CHECK DESIGN 40
6.3 EXECUTION ENVIRONMENT 41
6.4 PROPOSED QUESTION SET 41
6.5 BASELINES AND ABLATIONS 42
6.6 MEASURES 43
6.7 CONFIDENCE AND CALIBRATION 44
6.8 FAILURE AND RESILIENCE TESTS 44
Chapter 7 RESULTS AND DISCUSSION 45
7.1 RESULTS OVERVIEW 45
7.2 RETRIEVAL FINDINGS 46
7.3 ROUTING FINDINGS 46
7.4 EVIDENCE CONSISTENCY 47
7.5 INDEX CONSISTENCY 47
7.6 COMPATIBILITY AND REPRODUCIBILITY 48
7.7 REQUIREMENT ASSESSMENT 48

7.8 THREATS TO VALIDITY 49
7.9 INTERPRETATION 49
7.10 OBSERVABILITY AND MONITORING RESULTS 49
Chapter 8 CONCLUSION AND FUTURE WORK 56
8.1 CONCLUSION 56
8.2 IMMEDIATE IMPROVEMENTS 56
8.3 EVALUATION PRIORITIES 57
8.4 LONGER TERM EXTENSIONS 57
REFERENCES 58
Appendix 1 REPOSITORY INVENTORY AND OPERATING PROCEDURE 60
A1.1 SOURCE PROVENANCE 60
A1.2 PREPARATION 61
A1.3 RUNNING THE INTERFACES 61
A1.4 REPRODUCIBLE EXPERIMENT RECORD 62
Appendix 2 OFFLINE CHECK RECORD AND PROPOSED ACCEPTANCE TESTS 63
A2.1 EXECUTED ASSERTIONS 63
A2.2 PROPOSED ACCEPTANCE SCENARIOS 64
A2.3 ACCEPTANCE INTERPRETATION 65
Appendix 3 ALGORITHMS AND COST ACCOUNTING 66
A3.1 CORRECTIVE CONTROL ALGORITHM 66
A3.2 REQUEST ACCOUNTING 66
A3.3 RECORDING EXPERIMENTS 67
Appendix 4 SELECTED SOURCE LISTINGS 68
pipeline/retrieval.py 68
pipeline/verification.py 69
pipeline/generation.py 70
pipeline/query.py 72
pipeline/graph.py 75

LIST OF TABLES

Table 2.1 Research methods and their relationship to the project..... 20

Table 3.1 Functional requirement traceability 22

Table 4.1 Important graph state fields 27

Table 5.1 Source defaults and documentation differences..... 33

Table 6.1 Proposed comparison configurations 41

Table 7.1 Executed check groups..... 44

Table A1.1 Complete application file inventory 60

Table A2.1 Executed offline assertion record 63

Table A2.2 Proposed acceptance scenarios 64

LIST OF FIGURES

Figure 4.1 Application architecture derived from the source modules 25
Figure 4.2 Document ingestion and the two retrieval representations 26
Figure 4.3 Implemented corrective workflow 28
Figure 4.4 Conversation persistence model 30
Figure 7.1 Trace Count – Total number of successful and error traces over time49
Figure 7.2 Trace Latency – P50 and P99 latency percentiles over time 50
Figure 7.3 LLM Latency – P50 and P99 LLM call latency percentiles over time ...50
Figure 7.4 LLM Count – Number of successful and error LLM calls over time.....51
Figure 7.5 Total Cost – Cumulative API cost over time.....51
Figure 7.6 Cost per Trace – Median cost per trace (P50 and P99) over time.....52
Figure 7.7 Output Tokens – Total output token count over time 52
Figure 7.8 Input Tokens – Total input token count over time 53
Figure 7.9 Median Latency by Run Name – Per-component latency breakdown 54
Figure 7.10 Run Count by Name – Per-component invocation counts 55

SYMBOLS ABBREVIATIONS AND NOMENCLATURE

API Application programming interface

BM25 Best Matching 25 lexical ranking family

CLI Command-line interface

CRAG Corrective retrieval-augmented generation

DPR Dense Passage Retrieval

FiD Fusion-in-Decoder

LLM Large language model

ORM Object-relational mapping

QA Question answering

RAG Retrieval-augmented generation

REALM Retrieval-Augmented Language Model Pre-Training

RRF Reciprocal rank fusion

SDK Software development kit

UI User interface

k Number of retrieved results retained

rrf_k Rank-fusion constant, set to 60

m Number of local retrieval attempts

q Search query

d Document passage or chunk

Chapter 1
INTRODUCTION

1.1 BACKGROUND

A document assistant is useful only when its answers can be checked against the material it is supposed to explain. A fluent response is not enough. The answer may contain the right vocabulary while describing the wrong year, overlooking a condition, or confusing two similar terms. These errors become difficult to notice when a user has uploaded a long report precisely because reading it manually is inconvenient.

Retrieval-augmented creation solves this challenge by performing text retrieval before a language model begins to provide an answer. According to Lewis et al. (2020), a learnable model is developed. The present project uses the broader application. It involves splitting documents into smaller components, which are indexed, retrieved for their inquiry, and provided to a language model. The existing method does not train the first RAG design.

The name of the repository is Adaptive CRAG and it is a document chatbot built with Python and Streamlit interface as well as terminal interface. The repository makes use of embedding from Cohere, the dense vectors from Pinecone, keyword search from BM25 and the ranked results from relevance-ranked fusion algorithm. The repository uses Gemini for routing, query reformulation, evaluation of evidence, and generation of answers. The work of LangGraph allows connecting these functionalities together.

The repository named Adaptive CRAG is developed in Python and has both a terminal interface and a one built using Streamlit. The system uses embeddings provided by Cohere, dense vectors are stored by Pinecone, keyword retrieval is provided by BM25, and the results are combined using reciprocal rank fusion. The tasks required for routing, query reformulation, evidence validation, and answer generation are accomplished using Gemini. The technique that brings these processes together into a finite workflow is called LangGraph.

1.2 PROBLEM STATEMENT

While people communicate through the use of spoken language, the same subject may be expressed through technical terms and acronyms in different writings. Keyword retrieval can omit an effective passage owing to words mismatch. Dense retrieval can discover related content, but it may provide a passage without the exact fact that is expected. Therefore, none of the result lists confirm the absence of the answer.

In this case, the project is focused on the question of how the conversational document assistant can make use of semantic and lexical search, determine if the retrieved parts can answer the asked question, and compensate for the poor retrieval without creating an infinite number of calls to the model.

This is both an engineering issue and a language model issue. There's no use of a right ranking algorithm if the dense index has a document version that is older than that of the keyword index. The source list does not show that every sentence is supported. An evidence flag is only useful at the beginning, because as the process progresses, the documents might change without the flag being updated. The goal of this paper is to analyze the interfaces, as failures often happen in places where logic is not applied.

1.3 OBJECTIVES

The objective of this study is fourfold: to explain each application module and its function in the workflow; to compare mechanisms implemented with twelve existing research studies; to investigate deterministic behavior offline and assess the future of live evaluations; and to determine the improvements in evidence management, freshness of a corpus, compatibility, and separation of access. The published benchmark indicators are viewed as a background for this project rather than evidence of its success. The proposed amendments are well-distinguished from the unchanged database.

1.4 SCOPE

The document reports the latest version of the repository safal11goyal/CRAG, downloaded on October 12, 2026. The software comprises thirteen Python scripts, a list of dependencies, a README file, configuration files for Streamlit, a .gitignore file, and a PDF document. The PDF contains the World Health Organisation's Global tuberculosis report 2025, consisting of 76 pages. It is a valid example of a knowledge source, but not a labelled high-quality QA benchmark..

The app serves the function of an assistant with the text document. Having a report on tuberculosis does not imply that it is an authentic clinical system. The discussion refers to the document retrieval capability of the app, not to a medical recommendation. Questions provided later explain the retrieval methods rather than medical advice.

For this report, there has been no evaluation of Gemini, Cohere, Pinecone, Tavily, or PostgreSQL. There were no private credentials to be provided. As a result, the findings section mentions examining the source with some isolated offline tests. It does not provide any project

accuracy figures nor claims to have reduced hallucination rates or faked screenshots of the working implementation.

1.5 SIGNIFICANCE OF THE PROJECT

This project describes the process of retrieval. The process begins with the transformation of a question into a query. Then, a module gets potential texts from the database. After that, the verifier checks if the gathered texts are good or not. Lastly, the graph shows how to proceed. The way all these steps are divided helps to understand their code, and to future evaluators with identifying flaws in the project. It is worth mentioning the tradeoff in this design. While putting in more checks makes it less probable to make a mistake, every model-based check is time-consuming and inexact. Therefore, the evaluation of the system should be based on the quality of the entire decision-making process. Not giving a response is the right thing to do if there are no grounds for it and trying to make another search is unwise if the answer is clear already.

1.6 ORGANISATION OF THE REPORT

The subsequent chapters analyze previous research, specifications, design, execution, validation and the forthcoming tasks. The appendices contain various operational guidelines, examination notes and codes.

Chapter 2
BACKGROUND AND LITERATURE REVIEW

2.1 REVIEW APPROACH

This literature was chosen in order to address the key issues in the repository. Some fundamental papers were reviewed to showcase the role of retrieval in generation. Some works on retrieval and fusion were also studied to show how to present the candidate passages.

Some studies were devoted to corrective and adaptive methods, which show when the pipeline can modify the work after investigating the query or its proof. Evaluation work explains why a reasonable response is not enough in case we want to evaluate the quality of the result.

The use of major academic literature took place here, including conference proceedings, the ACL Anthology, papers provided by authors, and papers from arXiv. All twelve documents were gained in the format of full-text programs, and the necessary methods, approaches, and limitations were examined. The nature of this work is focused on engineering literature review and not intended as a comprehensive systematic review of the literature on RAG..

2.2 RETRIEVAL AUGMENTED GENERATION

Lewis et al. (2020) integrate a dense retrieval model with that of a sequence-to-sequence generation process and utilize retrieved documents as latent pieces of evidence. In a RAG-Sequence model, a document is used across an output sequence, whereas in a RAG-Token model, contributions of the document vary while generation is in progress. The authors conduct experiments using certain models for completion of knowledge-heavy tasks, including open-domain QA.

This paper is based on the idea of using external evidence for answering but accomplishes it differently. The system retrieves text through different APIs independently for the purpose of forming the Gemini prompt from several pieces of text. It operates without retriever-generator joint training, document marginalization, and RAG-Token decoding. The paper offers some theoretical basis but does not set out the results for the implemented system.

On engineering level, the one comes to the conclusion that both retrieval and generation have to be accessed separately because the generator can correctly produce a text for a wrong passage thus giving a wrong answer to the query. Thus, the paper introduces the evidence decision that exists between retrieval and generation and requires evaluation of its own.

2.3 DENSE PASSAGE RETRIEVAL

Karpukhin et al. (2020) illustrate a dual-encoder method for passage retrieval that relies on open-domain methods. Distinct embedding is utilized for questions and passages allowing the passage representation to be efficiently precomputed. The authors' training method consists of comparing positive passages with those that are irrelevant to the inquiry. The conducted experiment assesses how improvements in retrieval affect questions answering.

In the current work, Cohere substitutes DPR encoders allowing Pinecone to provide the vector index. The framework does not repeat training of DPR nor the testing results shown in the paper under consideration. The significance of the suggested approach derives from the capability to separately track the document and the query embedding processes respectively.

Such separation creates a necessity for compatibility. Query and document vectors need to be generated in the same embedding space, while probes do not guarantee the compatibility of the models. In order for the method to be successfully applied in practice, the configuration of the index needs to include both the identifier of the embedding model and the corpus version.

2.4 RETRIEVAL AUGMENTED PRETRAINING

Guu and colleagues (2020) presented REALM, leading to the retrieval learning that makes use of the masked language model technique and involves the fact that the documents are retrieved as hidden variables. This method combines knowledge retrieval, pretraining, fine-tuning, and inference. Thus, the problem of the successive modification of sets of representations during retrain process has been solved.

The basis of adaptive CRAG is hosted models that exclude the training stage. REALM has some similarities with these types of models as they are also capable of separating the external knowledge from the parameters of models.

Speaking about this application, it is important to mention the necessity of the index maintenance. The current initialization procedure checks whether some vectors are available but does not check whether they are similar to the documents existing in the moment. The future version of the application should possibly check the currency of the corpus before providing the answer..

2.5 RECIPROCAL RANK FUSION

Cormack et al. (2009) put forward the concept of reciprocal rank fusion, which involves determining contributions made by the ranks in terms of their inverse value for each system.

The method does not presume that outputs generated from different systems use similar scale for scoring the output collections. The present study concerns the TREC collections as well as the LETOR rankings, where the rank constant has been set at 60 during the research.

This repository employs the following basic formula for combining two lists, that are dense results from Pinecone and BM25 results. The contribution of the passage is counted twice if it appears in both lists. The identity of the passage is established owing to the trimmed text, origin, and metadata of the page from which the passage originates. The fused output list is filtered according to the retrieval limit imposed.

The authors advocate an easy baseline for fusion, but they fail to demonstrate that equality of weights is optimal in this instance. The situation should be examined in the light of the first two cases described above as well as the alternative fusion in the setting of dense systems, sparse systems, and other types of hybrid approaches.

2.6 FUSION IN DECODER

Izacard and Grave (2021) examine how a generative reader can merge several retrieved segments of text. Fusion-in-Decoder processes the segments of text separately and allows the decoder to utilize multiple representations of these segments. The findings of the research indicate the advantages of using more evidence in question-answering tasks.

The generation function of this project, on the other hand, integrates the passages in a single context. Thus, the approach does not employ Fusion-in-Decoder. The advantage of this comparison is that it allows the differentiation between retrieval of multiple pieces of text and the task of text combining through employing specialized architecture.

For this project, the result of increasing retrieval counts can have positive impact on evidence density, but it can also lead to enlarged input and distractors. Therefore, an experiment with retrieval counts should help understand the efficiency of the model in retrieving or supporting the answer.

2.7 SELF RAG

Asai et al. (2023) present Self-RAG which trains the system such that it can send reflection tokens used for retrieval and assessment of the quality and usefulness of retrieved information. With these signals it is possible to conduct the inference in a more precise way than just using a single retrieve-answer event. The considered manuscript in arXiv contains the data on the experiments conducted for both short and longer tasks.

The repository has a separate verifier with a prompt and a conversational router. The system does not perform any training on the reflection tokens and does not use the inference of Self-RAG for scoring the numerous generated alternatives. It can be called a reflective project in a broad sense but can't be regarded as an instance of Self-RAG.

The comparison shows that an important difference that is missing in the current design is the analysis of the context before generation from the one of checking whether the generated statement stems from it. A verification carried out after the generation is over could fill this gap without suggesting that the idea of the complete Self-RAG system has been learned..

2.8 CORRECTIVE RETRIEVAL AUGMENTED GENERATION

Yan et al. (2024) study failures caused by poor retrieval. Their CRAG method evaluates retrieved documents and selects correct, incorrect, or ambiguous actions. It includes a trained retrieval evaluator, knowledge refinement through decomposition and recomposition, and web search when additional evidence is needed.

The project adopts the corrective idea but simplifies the mechanism. Its verifier returns a Boolean sufficiency decision for the retrieved context. Failed local attempts trigger query rewriting, and exhausted attempts trigger Tavily search. It does not implement the paper's three-way confidence categories or knowledge-strip refinement.

This distinction matters when interpreting the project name. The repository is an application inspired by corrective retrieval, not a benchmark reproduction of Yan et al. Its effectiveness must be measured on its own corpus and questions. In particular, the unchecked web branch is an implementation-specific limitation rather than a property established by the paper.

2.9 ADAPTIVE RETRIEVAL

Jeong and others (2024) have come up with Adaptive-RAG, and its core idea lies in the fact that the accessibility of retrieval techniques depended largely upon the complexity of questions put before it. Basically, the whole classification of questions happens based on whether questions require retrieval or not, whether they need to be dealt with using standard retrieval or more complex retrieval systems. The authors of this particular paper analyze different question types and their pointing towards various retrieval systems they might be using.

The differentiation in the approach of this project is limited. It separates conversational messages from those requiring retrievals. Unlike Jeong et al., it does not utilize a classifier for identifying the types of questions.

2.10 QUERY REWRITING

Research by Ma and colleagues (2023) investigates a Rewritten-Loaded-to-Read process. They analyze the possibility of using a trainable rewriting model that learns from the users' feedback after providing them with the possibility of prompting a search request.

The application is utilizing the idea of rewriting twice: first, to transform a question into a search query and then after that upon failure to verify the information source. The second elicitation uses the same question, the previous request, and the explanation of why the verification did not work.

The implemented technique is easy to use anew, however, using the rewriting approach might change the year, the name, or the relation requested. Consequently, the performance assessment must be carried out in such a way that these changes are revealed.

2.11 CONTEXT POSITION AND LENGTH

According to Liu et al. (2024), the focus of their research has been to understand how information is processed in different positions of long texts in language models. Different tests were carried out, based on multi-document question answering and key-value retrieval. Depending on the position, performance was affected since the constructive usage of context and context capacity are not the same concepts.

This is applicable to the sequence of passes in the project. In terms of RRF, the elements that are included in the prompt and their order are straightened out. Despite that, the implications discovered in the paper cannot be taken for granted using the developed Gemini model.

What could be done in the experiment is to conduct an experiment similar to this one, but to make some changes in the order of the passes. In such a way, the retrieval accuracy will be distinguished from the usage of context by the reader..

2.12 AUTOMATED RAG EVALUATION

Es and co-authors (2024) discuss RAGAs, which is an assessing structure for determining the relevance of facts, answers, etc. The work investigates the system of automated evaluations and their parallels with human judgment. Its shortcomings include reliance on the assessing model and validity of its evaluations in different situations.

The dataset doesn't contain RAGAs assessment pipeline, although its confidence category is formed in the process of answer generation and is not an independent evaluation criterion.

RAGAs are applied as an assistance for the proposed verification technique rather than the pre-existing function of the tool.

A practical evaluation should include questions, contexts, answers, as well as human-reviewed tags so that it is possible to identify retrieval failure and the lack of support of generation processes. Automated evaluation may help in this respect but a small piece of manually verified data is a must for spotting mistakes made by the evaluator..

2.13 CITATION QUALITY

ALCE has been proposed by Gao et al. (2023) to measure the performance of produced responses that comprise citations. The mechanism takes into account parameters such as fluency, correctness, and citation quality, which addresses if the citations back up the statements. The study shows that merely providing citations isn't tantamount to proof that supports claims.

The current analysis lists the names of all of the sources as well as the corresponding page numbers obtained. Yet, there is no mapping of claims and relevant text blocks here. Thus, the provided sources do not prove validity on the level of sentences.

The above-mentioned difference suggests a practical enhancement where one should give a stable citation identifier for each piece of text, indicate it among identified factual claims, and check their validity.

2.14 SYNTHESIS AND RESEARCH GAP

These research papers face the process of retrieval as a multi-faceted concept requiring various decisions like if to initiate a search, the nature of the search query, the number of contents to be retrieved, how to combine the information, and how to trust the search results. The repository makes it possible to use these aspects as different functions and graph vertices.

The main aspect that makes it attractive for research is not the addition of new components but testing the existing corrective loop to determine whether it can improve results compared with a certain baseline when index consistency and evidence handling are controlled.

One-pass hybrid and complete corrective models should deal with the same questions and collection. The responses should then be compared in terms of their validity, completion, service use, and latency. The studies in the field do not give the universal answer regarding the size of chunks, parameter k, number of retries, and the prompt of verifier. The aspects depend on the corpus and the questions..

Table 2.1 Research methods and their relationship to the project

Research line	Project relationship	Important distinction
RAG and REALM	External knowledge informs generation	No joint training or retrieval pretraining
DPR and RRF	Dense retrieval and rank fusion	Hosted embeddings replace DPR training
Fusion in Decoder	Multiple passages inform one answer	No separate passage encoder architecture
Self RAG and CRAG	Retrieval decisions and evidence checks	No reflection-token training or three-way evaluator
Adaptive RAG and rewriting	Routing and query reformulation	No learned complexity classifier or rewriter
Context use and evaluation	Motivate controlled quality assessment	No benchmark suite is supplied

Chapter 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 EXISTING APPROACHES

The keyword search tool can disclose useful material without giving a response. However, it is somewhat limited, as the user must look through the material received. The major disadvantage is that the user must do the hard work especially when an answer involves a few excerpts. The issues of incorrect use of words complicate the whole process since the exact words must be used.

The assistant based only on a language model is easy to communicate with but has nothing to do with the documents included in the prompt. It can provide answers based on the knowledge it has even when the question is directly related to the report version. It is especially dangerous when the question presupposes an implicit limitation "as per this report".

A one-pass RAG system has simplified retrieval of documents related to the inquiry. The system retrieves the information only once and then provides the answer using the context it has. The problem is that the quality of retrieval depends on the question. In some cases, even the best passage does not give the required answer. To solve this issue, the project offers to verify the information and create an alternative solution.

The corrective mechanism poses its own threat. An error in either the verifier can cause useful passages to be eliminated, a revised version may deviate from the focus of the question, and a web search may get rid of the line of evidence one wanted to use. Such points are not enough to avoid using them. Instead, one must take into account the risks associated with each step and make the origin of the sources traceable.

3.2 FUNCTIONAL REQUIREMENTS

The ingestion subsystem is supposed to read the local documents that it supports, extract contents, retain source metadata, and produce overlapping chunks of extracted contents. After the chunking process is over, pages of PDFs and presentation slides must be still traceable. In case of unsupported or unreadable files, there must be a meaningful error shown. Loading of the folder should not be hindered by the failure of the loading of each particular file.

The subsystem for retrieval should accept a search query, derive dense and sparse lists of candidates, merge their ranks, and produce no more results than required. Handling of empty

keyword collections should be possible. The output must have enough metadata that satisfies the purpose of source formatting and further analysis of errors in the retrieval process..

The orchestration system must be able to differentiate conversational questions from professional inquiries, process the inquiry, evaluate the facts, and restrict attempts to retrieve locally. It must also facilitate two kinds of searches: web search and giving a clear statement of having insufficient proof. It must also make the original user inquiry accessible even when the search is carried out using a different expression.

The response subsystem must generate an answer while giving a confidence value, a caution, and information about the source. Persistent storage of conversations is optional, but if in use, the subsystem must store the question, final search string, answer, evidence flag, verification reason, and number of calls. A temporary database malfunction must not be considered as strong evidence for failure of the query.

Table 3.1 Functional requirement traceability

ID	Requirement	Implemented location
FR1	Load and split documents	pipeline/ingestion.py
FR2	Store and search embeddings	pipeline/pinecone_store.py
FR3	Combine dense and keyword results	pipeline/retrieval.py
FR4	Route and rewrite queries	pipeline/query.py
FR5	Assess local evidence	pipeline/verification.py
FR6	Bound corrective attempts	pipeline/graph.py
FR7	Search the web when needed	pipeline/web_search.py
FR8	Generate and format sources	pipeline/generation.py
FR9	Retain conversation turns	database.py
FR10	Provide web and terminal access	app.py and main.py

3.3 QUALITY REQUIREMENTS

Traceability refers to the ability to go back to the original document from which information was taken from the displayed URL. Data retrieved from the website contains reference data that help trace back to the source. Additionally, there should be information about the chunk

ID, corpus version that was used, as well as query history and ranking data since special information is required to track changes in legal documents in the future.

Reliability indicates that the failure of one service should have a predictable outcome. The system is able to find errors in verification, generation of information, web searching, and database operations, but it does not use a common error framework for the entire process. In the optimal situation, it should differentiate between the situation when a service is absent, when a necessary input is not retrieved, an answer is improperly formed, or the system is misconfigured..

The requirement of responsiveness means that a number of calls and clear responses have to take place. The graph indicates the limit of the efforts taken with the locality while the user interface gives the result expressed in words during the time of completion.

The last mile is influenced by many factors like calls and requests made to the models, embedding, vectors, and others. All these expenses need to be strictly determined instead of deducing them from the graphic presentation shown by the graph.

Maintainability benefits a lot from modular files and typed schemas of output. When it comes to reproducibility, it is slightly less strong since dependencies are not locked, but treated as lower bounded ones. People cannot reach any consensus about the definition of several options found in the README..

3.4 DATA AND CORPUS REQUIREMENTS

The provided PDF clearly illustrates a real-life type of document: a long report with introductory pages, numbered text pages, tables, charts, references, and attachments. Extraction of text from such a document is not perfect. A chart name can be dissociated from a number, a table can lose its row composition, and the number of the physical PDF page may differ from the printed page number on the document.

The loader has physical page indices starting with one for its PDF documents. Although this solution works for finding a page from a perspective of a viewer, the end user should not assume that this page index will be aligned to the printed page index in the report. The metadata design of the future can have both indices stored in the same format. The same can be said about presentation graphs where slide numbers can be extracted as in the regular page index field.

The corpus does not contain a supplied set of expected answers or relevance labels. Consequently, the PDF establishes document availability and extraction coverage, not

benchmark accuracy. A valid evaluation dataset must be created separately, with questions tied to verified evidence spans and a record of which questions are intentionally unanswerable.

3.5 FEASIBILITY

The prototype is technically feasible by virtue of its nature as an API-based implementation, where embedding, vector search, text generation and web search operations are handled by third parties, while the local code focuses on orchestration and presentation. This decreases model training requirements, but increases requirements for network access, credentials, service compatibility, and limits.

The operational feasibility of the application relies on how ocuments are managed and who owns them. The convenience of a single local folder and single vector namespace is not sufficient evidence that the users are separate in a multi-user deployment of the system. Prior to accepting private documents, the system should include a policy that specifies which

user can retrieve chunks of information.

Economic feasibility of the application cannot be estimated solely on the basis of this repository. There are no request and token statistics. However, the implementation can still be examined in terms of the additional requests it makes verification and rewrite require model calls, ingestion requires embeddings, while web fallback requires one search request. Deployment analysis should estimate the cost using actual service usage statistics for a fixed workload.

3.6 ASSUMPTIONS AND CONSTRAINTS

The embedding system used is one that operates in English, so this report does not assume that the efficiency will be similar across different languages. When the document folder is loaded, no recursion takes place. The program is intended for use with texts and does not include the OCR stage. Consequently, scanning PDF files would yield no acceptable results even if the files appear correctly in a viewer.

The configuration parameters may be entered, but they cease to be relevant to the essence of validation. For example, the overlap must remain less than the chunk size and the retrieval and iteration parameters must be positive. These restrictions are not displayed as field limits in the settings class of the system.

The system treats the names of the external models as configuration parameters rather than as formal proofs of their existence; the original model is referred to as "gemini-3.1-flash-lite",

even though the README states that it is "gemini-3.5-flash-lite"; in order for the system to function correctly, it must verify that the provider supports the model.

Chapter 4
SYSTEM DESIGN

4.1 ARCHITECTURAL OVERVIEW

The system excels because of its two entrance points and the same pipeline as well. Streamlit enables document upload, chat messages, screen of the sources, and conversation choice. The terminal app features a command mechanism and command history. Both make embedding and language model clients, get vector repositories and local chunks, build BM25 index and form LangGraph process.

The knowledge layer has two versions of the corpus. Pinecone stores vectorized data and text data remotely. BM25 stores tokenized chunks in RAM. These two data types are merged only for the purpose of query execution. Therefore, it is important to make them relevant time-wise. Using one interface does not eliminate the risk of having different version of files in these two stores.

The control layer is a directed graph with shared edges. The information moves from one node to another in the form of a dictionary referred to as RAGState. The results layer obtains the final state and displays its parameters.

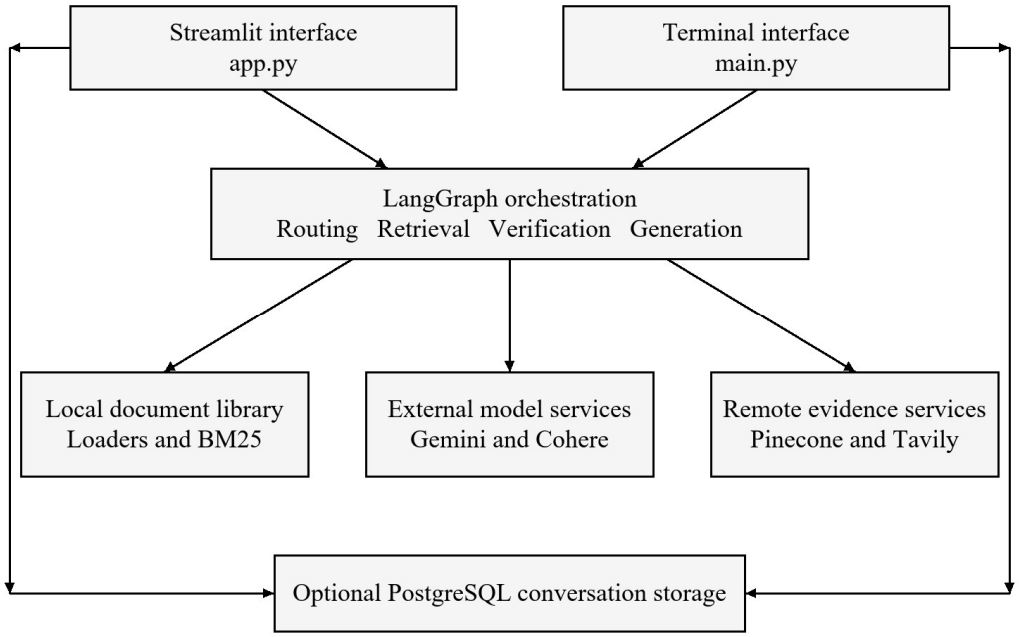


Figure 4.1 Application architecture derived from the source modules

4.2 INGESTION DESIGN

Filing starts in the designated documentation folder or when receiving bytes via the web interface. The loader starts with rules for disconnecting by extension. The loader for PDF connects the pages with their indices. The loader for the presentation goes through the slides and extracts characters and words from different shapes and tables. It also includes the presentation speaker's manual, wherever it is available. The loaders for TXT and DOCX load the files with the filename being the only source of reference, without creating captions for the pages.

The extracted files are then processed by the RecursiveCharacterTextSplitter, breaking them down into segments of text that overlap. The document itself has some sources in the form of paragraphs, lines, white spaces, and characters added. As no character-counting rule is established, the limitations are based on the number of characters.

The startup path retrieves the local files and splits them into chunks before establishing a connection with Pinecone. It checks the embedding dimension, establishes connection to or creates its own index, and finds out the vector count. If vectors are already in existence and force_reindex is false, it exits with the local chunks in place immediately. Otherwise, it encodes and upserts batches. This avoids the necessity to carry out the process of ingestion constantly but does not guarantee a comparison of the corpus.

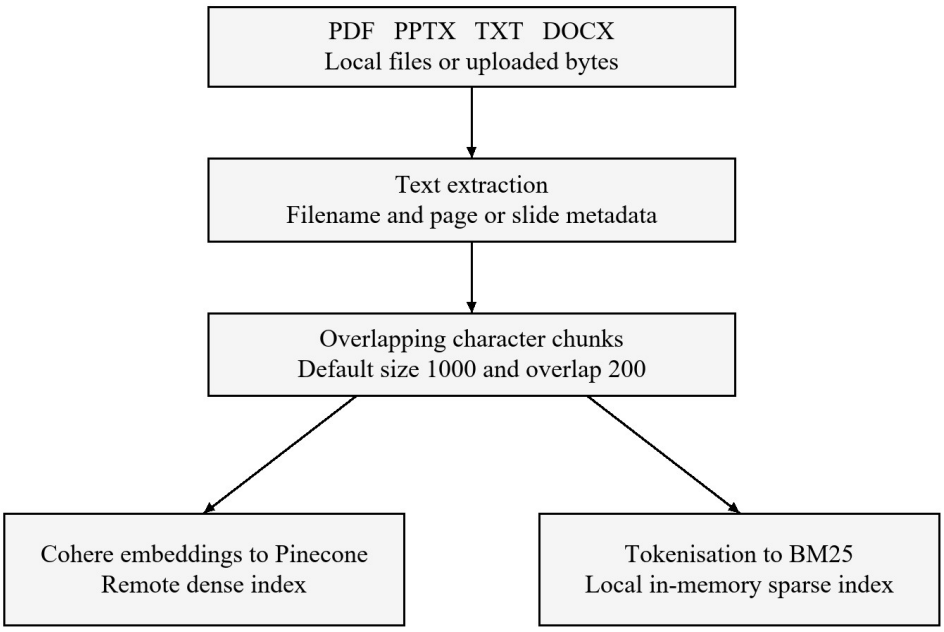


Figure 4.2 Document ingestion and the two retrieval representations

4.3 QUERY STATE

The original question is kept aside by RAGState from the search query. It is critical to note that while phrasing of searches may alter, the same is not a case with evidence verification and response. Use of a summative writer in the messages column allows for adding assistant messages by the response nodes.

The documents column has the latest retrieval results. Replacement of that by the amalgamated fragments occurs with local retrieval while web search replaces that with the results of the web. The evidence is not collected through local tries by the graph. No matter it was discovered during an earlier attempt, evidence disappears from the state unless found again in the latest output.

Fields enough_evidence and verification_reason contain information relating to the latest local verification. They do not change even after web search. The confidence column has two options: either to be a component of an answer or a permanent branch-dependent parameter.

Table 4.1 Important graph state fields

Field	Purpose	Update behaviour
question	Original information need	Retained across correction
search_query	Current retrieval wording	Replaced by query nodes
documents	Current context passages	Replaced by retrieval or web search
iterations	Local retrieval attempts	Incremented by retrieve node
enough_evidence	Local sufficiency decision	Updated by verifier or direct chat
verification_reason	Explanation of local decision	Latest verifier result
answer and sources	User-facing output	Set by terminal response node
confidence and warning	Model estimate and route notice	Different semantics by branch
messages	Conversation context	Assistant response appended

4.4 CONTROL FLOW

The first node determines whether retrieval is required. A conversational message arrives at direct_chat and then stops. A factual message passes through understand_query, retrieve and

verify_evidence. If the local evidence is enough, a response is generated. If the evidence is not enough and the counter is below the limit, rewriting occurs; otherwise, web_search takes place.

The counter is incremented during local retrieval. The graph allows three local retrieval attempts and only two correcting rewrites after initial query optimization with a default maximum number of three attempts and an initial count of zero. This cannot be described as three rewrites because the loop, as implemented, is more complex.

After web search, it is necessary to check whether or not there are any documents. If there is a non-empty list, it goes for generation, whereas an empty list goes to the insufficient-evidence handler. This path doesn't allow for a second verify_evidence call to take place. The flow diagram shows that this path exists because in case of adding a verification step in the diagram, the code would not be depicted accurately.

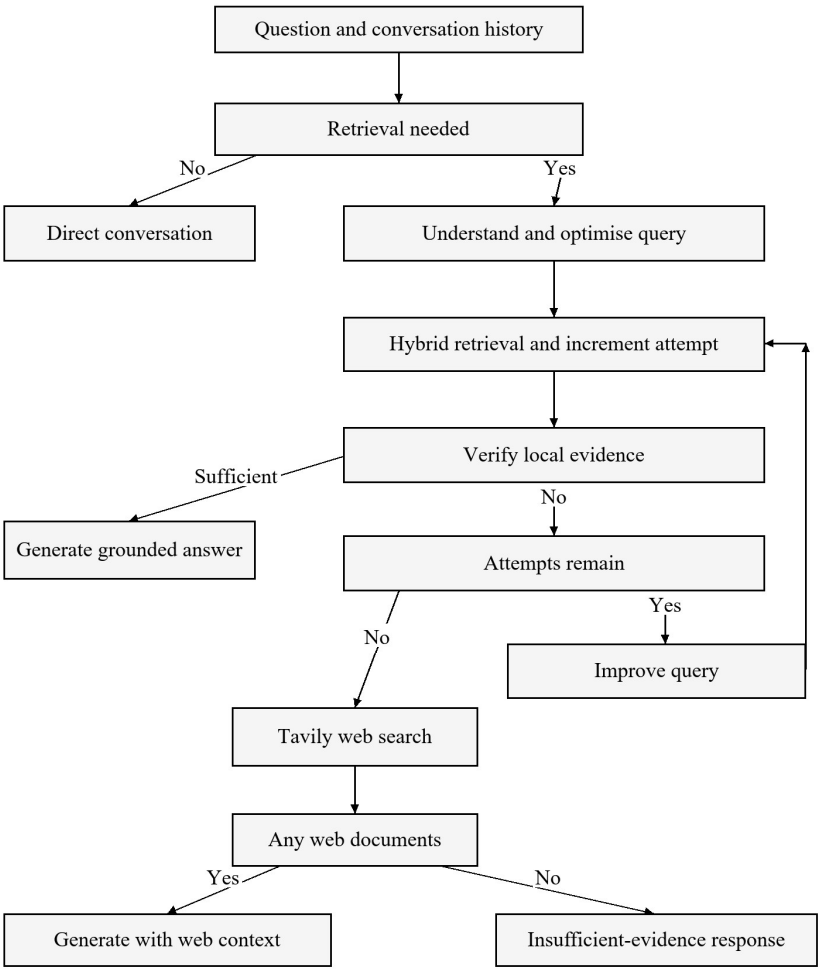


Figure 4.3 Implemented corrective workflow including the unchecked web branch

4.5 HYBRID RETRIEVAL DESIGN

For dense retrieval, the input query is embedded into vectors and the nearest retrieved vectors are sought from Pinecone. In the case of sparse retrieval, the input query tokens are searched through a BM25 corpus. Both operations are executed sequentially in hybrid_retrieval, as their execution is not parallel despite their conceptual independence.

Each output from the hybrid retrieval process is assigned a fusion key that consists of the cleaned passage text, the source, and the page. This ensures that even if the same sentence appears in different files, it is still traced to its original source and accounted for in the output ranking, ensuring that the outputs from dense and sparse retrieval are merged together. Each output contributes a value of 1/60 plus its rank output.

As an example, if the output of dense retrieval ranks A higher than B and that of sparse retrieval ranks B higher than C, the score for B will be equal to 1/62 + 1/61 = 0.03252 and for A will be equal to 1/61 = 0.01639 and finally for C is equal to 1/62 = 0.01613. Therefore, B is ranked higher than any other output in the process; these scores are not the final scores but simply necessary values for writing the equation.

The technique utilizes positions in place of raw similarity values. This will mean that BM25 scores are not directly combined with the cosine similarity. This also means that even a dismal dense match will add to the ranks. There is no specific threshold for dense similarity in hybrid_retrieval. Instead, evidence verification is expected to help detect inadequate context after the fusion stage.

4.6 EVIDENCE AND RESPONSE CONTRACTS

EvidenceVerification has two components: the amount of evidence and reasoning. The task requires a more rigorous examination of how to determine if the prompt has specific information required for answering it. The schema specifies the appearance of the output data, but it does not guarantee the correctness of the answer.

The AnswerOutput section contains information about the answer, verdict, and caution. The engine is instructed to use only the evidence provided to produce the output. The program restricts the verdict to the range of values between zero and one and sets a basic low checklist rulings here. These are the rules for generating output but they do not ensure accuracy or any evidence.

More detailed contracts for evidence would improve the project even further. The utilization of Boolean, subquestions supporting it, missing facts and references to the texts could be a better solution.

4.7 DATABASE DESIGN

There are two main components of the database, which are conversations and conversation_turns. A conversation has string type session identifier, title (which can be absent) and date of creating and updating born with it. Each turn is connected to conversation through foreign key and stores the question from the user, last search query, answer, sources, evidence flag, verification reason, confidence level, warning and the number of the iterations.

This relation allows one conversation to have many turns. The ORM relation states delete-orphan cascade of turns and foreign key makes a cascade delete at the level of the database. There is no deletion interface, which means this feature does not allow us to say that it deals with the retention management.

The only thing which is saved is last search query and last verification reason. Other intermediate versions are not saved. Thus, from the database we can see that there were several attempts of retrieval, but we do not know what was different. Thus, it would be necessary to develop the loggings.

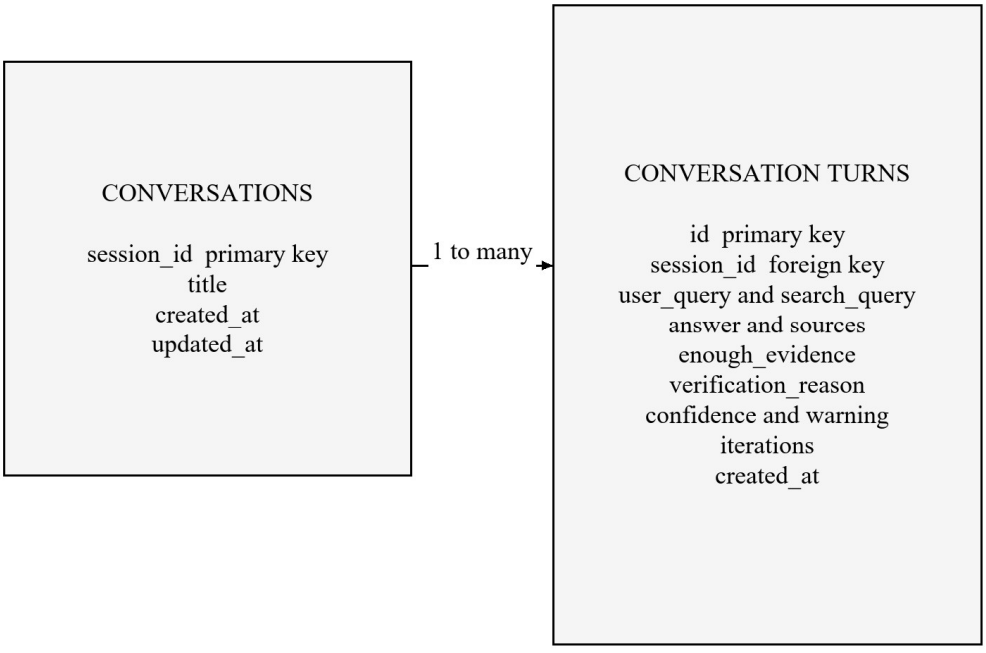


Figure 4.4 Conversation persistence model

4.8 INTERFACE DESIGN

The Streamlit sidebar consists of controls that enable chat creation, file uploads, refreshing the library, and previous chats browsing. The chat messages and a source/ground expansion are displayed in the center when they appear. The sidebar reveals the chosen model and embedding model, vector index, and retrieval count.

The interface shows the completed answer letter by letter after graph invocation. It does not stream tokens from the model. Such a difference leads to perceived latency in waiting for the entire graph because the user has to start visualizing only after all tokens are sent. The future design of streaming should make sure that structured output and warning handling are preserved while partial output is received.

When the database is not available, Streamlit keeps messages only for the session. This is temporary continuation of the conversation instead of fixed database tools. The terminal interface also keeps messages in memory although its history command calls the database.

4.9 DESIGN ALTERNATIVES

A cross-encoder reranker can be utilized after the completion of fusion, to assess the pairs formed from the question and passage more precisely. The advantage of using it would be in terms of better order or elimination of other bad passages; however, its drawback would be extra inference. It must be used after the baselines have been tested in advance for dense-only, sparse-only, and fusion methods.

The local embedding model and local vector database, on the contrary, would limit the external dependencies but would require different deployment and resource-related decisions. The current modular wrapper offers the chance of substitute of local embedding model with local vector database. Therefore, the use of such methods remains plausible; however, it is not automatic. Compatibility tests are still the necessity for indexing, metadata, retrieving order, and failure issues.

Another strategy for correction can be local with web evidence. In the current version, local evidence is replaced with fallback entirely. However, the mix of the sources can have positive and negative consequences as well. Thus, any extension must fulfill the requirement of keeping provenance and be able to explain how certain evidence make it possible to support claims made.

Chapter 5
IMPLEMENTATION

5.1 ENTRY POINTS AND INITIALISATION

main.py ensures an appropriate API configuration, and validates the presence and non-empty state of the documents directory. Also, it performs database initialization and creates the model clients. Afterward, it either loads or creates a vector store, and builds the BM25 index, followed by the graph construction and subsequent switch into an interactive mode. The commands /history and /sessions retrieve the stored conversation data. The exit, quit and q commands stop the session.

app.py implements the similar process of the pipeline construction via a cached resource function. The cache eliminates the need of pipeline building after each Streamlit run. The uploaded files are ingested, with successful uploads deleting the resource cache prior to the application rerun. The cache level of operation is an important point when there are several users using one process and knowledge base simultaneously.

At the same time, entry point web does not call validate_config the way it is called in the terminal. Therefore, failures can occur at the stage of client creation or ingestion rather than in the messages of validation generated by the CLI.

5.2 CONFIGURATION

The config.py file utilizes pydantic-settings to acquire data from the environment variables and a local path of .env file for the project. Once imported, the settings will be used and a constant is established at this point. Config will provide a basic technique for the modules that import it.

The default in regard to the source consists of the following: chunk size of 1000 symbols, overlap of 200 symbols, retrieval of four documents, and at least three document retrievals. The model identifier is as follows: gemini-3.1-flash-lite. The default embedding identifier is: embed-english-v3.0. The vector index name is set to rag-index. AWS and us-east-1 are provided as creation parameters..

A provided PostgreSQL URL is normalised by the database helper via SQLAlchemy's URL parser. In case the full URL is not available, a URL is created from separate environment variables. The said alternative directly substitutes username and password, which leads to the need that particular reserved characters of a password need to be taken into account. A more

reliable version of a backup solution is to use URL object API for constructing the backup URL.

Table 5.1 Source defaults and documentation differences

Setting	Source value or behaviour	README difference
GEMINI_MODEL	gemini-3.1-flash-lite	Names gemini-3.5-flash-lite
RETRIEVAL_K	4	Shows 5
CHUNK_SIZE	1000 characters	Described as tokens
CHUNK_OVERLAP	200 characters	Presented without correcting token wording
MAX_ITERATIONS	3 retrieval attempts	Some wording suggests rewrite count
Streaming	Completed answer displayed incrementally	Described as real-time streaming
Tests and licence	No tests directory or LICENSE file in snapshot	References both

5.3 DOCUMENT LOADERS

load_single_document verifies whether a path exists and whether it is a file. For loading a PDF, PyPDFLoader is utilized, the source metadata is replaced by the file name, the page numbering is switched to a basis of one, and the pages are dismissed which do not possess any content apart from spaces. This maintains the slimness of the corpus, but it may not show up the image-only pages unless their extraction is checked separately.

python-pptx library is employed to load presentations. The process involves tracing through text boxes, tables, grouped shapes, and speech notes. Any slide, which is not empty, becomes Document which retains the meta data of the page. The extension manager accepts both .ppt and .pptx files, although the underlying parser cannot read legacy .ppt files, so the user is instructed to convert them in case of an error.

Plain text files are encoded with UTF-8 using the automatic detection of their encoding. In the case of DOCX files, Docx2txtLoader is used. The library for the dependency contains the python-docx library and does not directly mention docx2txt.

The loading of directories allows for the arrangement of files in a consistent manner but only processes the parent directory. Any loader exceptions become warnings, which do not hinder

other files from being processed. This is beneficial for redundancy, but a successful running of the program must also indicate which files were missed so that one does not mistakenly think that the entire directory has been indexed.

5.4 CHUNKING AND INGESTION

chunk_documents is responsible for working alongside RecursiveCharacterTextSplitter, while also utilizing paragraph, newline, space and character delimiters. Each chunk created retains metadata. Although overlapping chunks will make it easier for processing context, it results in repetitive information in adjacent chunks. It should be taken into account during retrieval evaluation.

The size of each ingestion batch is 30 documents. Every batch executes add_documents in the wrapper for the vector store. A Tenacity retry policy once more works with recognized rate limit issues allowing to repeat this as many as eight times while utilizing an exponential backoff mechanism that confines it to 10-80 seconds. The recognition process has to deal with certain keywords such as 429, rate limit and resource_exhausted which are very helpful, but depend on the provider's wording..

After processing a batch, the startup ingestion loop pauses for a period of three seconds. The upload loop pauses only between the batches. These pauses are implementation decisions and are meant to reduce pressure on the rate and, as such, do not guarantee compliance with any current provider quota. Confirm the required rate policy based on the current workload and account configuration.

ingest_uploaded_file writes the received bytes to the file prior to parsing them. Hence, a parsing error may leave a file in the library directory. The number of files that are visible refers to local files and is not an indicator of the success of vector indexing. It would also be useful to extend the next ingestion record by separating the documents based on whether they were received, parsed, indexed, failed, or superseded..

5.5 VECTOR STORE WRAPPER

PineconeVectorStore is able to add documents and search for similar documents thanks to the LangChain VectorStore interface. add_documents creates a stable id using source, page, and content. It takes the first 32 characters from a SHA-256 digest to serve as the identifier. Obtaining the same identifier while performing insertion means that the idempotent operation replaces an existing record instead of creating a random record.

The idempotent insertions are limited to unchanged parts of the document only. Changes in text lead to the generation of a new identifier. Old vector removal is not handled by the code automatically. Thus, a replacement of a local file may lead to outdated elements in remote storage without implementing a separate deletion/version policy.

The cleaning of metadata maintains situations involving strings, numerical information and lists of strings while also dropping null values, leaving other values processed either as strings or in the form of lists containing strings. The metadata does indeed contain the text of the full text connected with the corresponding document. When called, search reconstructs Document objects from the metadata and pulls the appropriate similarity scores, if requested.

The wrapper identified accepts both a namespace and a metadata filter, although the application in question simply relies on the default empty namespace and does not apply any user-specific metadata filter through hybrid_retrieval. This means that the capabilities offered by the wrapper are merely extension points rather than a demonstration of tenant isolation being implemented.

5.6 KEYWORD RETRIEVAL AND FUSION

BM25Index is a text tokenization tool that divides text into tokens using a regular expression for lowercase letters. A BM25Okapi object is created whenever the corpus is not empty. Queries are tokenized in the same manner, and results are sorted according to their scores which are only positive. Empty corpora and empty tokenization results will yield no keywords.

The tokenization process carried out by this particular tokenization tool is quite simple, reproducible and straightforward as it does not include any stemming, special handling of abbreviations or phrases matching. The embedding path could be helpful with the vocabulary and it should be tested against some sample queries. Also, technical acronyms and report-specific terminologies could be used to test this technique.

The RRF function takes the results and generates a dictionary key according to it, being attached to a score calculation. In case of a tie in scores, the insertion order can affect the score outcome. Dense candidates must be first taken into consideration, but it is more of a minor detail rather than an issue to worry about.

5.7 QUERY ROUTING AND REWRITING

check_retrieval_needed first checks the greetings and basic conversational phrases. When there is an exact match, the model does not perform a request. Then prefix rules for phrases such as

“my name is,” and “I am” are applied. All other messages are sent to the model that generates the replies for yes or no. If this model call fails, the function returns True and tries to retrieve the documents.

Prefix rules are too extensive for some factual queries. For example, the sentence “I am doing research on tuberculosis incidence” matches the prefix, thus avoiding retrieval. This behavior was tested offline. The more appropriate approach would be to narrow down the recognition of the introduction or allow the intent model to look at longer messages and understand the actual question.

understand_query gets a slice of messages sent before the current question and forms a simple search query. improve_query accepts the original question, the previous query, and the verifier's reason. Both functions strip the simple quotation wrappers and go back to the original or previous query in case the parsed output is empty. The exceptions created by the models are not solved locally in these two functions..

A problem related to maintaining consistency during the interaction begins when one is able to comprehend the question. The verification and the generation proceed with access to the initial wording of the inquiry but without the chat exchanges or the solved question. For instance, a follow-up question like "What is going to be used to finance this?" could suffice at the retrieval stage, but become less implied at a later stage. Therefore, retaining the simple question within the factual branch may help solve this issue.

5.8 VERIFICATION AND GENERATION

More specifically, verify_evidence provides an instant negative result if there are no documents. Otherwise, it processes the source labels and passage text, requests a structured EvidenceVerification output, and returns the resulting object. It is noteworthy that the conversions of exceptions into negative evidence results happen along with the reasons for the error. In this way, the mechanism avoids confirming evidence while the verification fails; however, it has to deal with the failure of the service as if it was an insufficient evidence for the purposes of routing.

generate_answer operates the same way although it has to do the formatting of the context and ask for AnswerOutput. It tells the model not to use any information outside the available data. After parsing, the mechanism sets a limit for confidence and ensures that there is a warning. If it fails to generate the result, it provides an error with the level of confidence equal to zero and a warning about the risk of failure.

format_sources also removes duplicate page numbers from their strings. It does not generate any page numbers for DOCX or text sources. Nevertheless, it includes all retrieved documents, even when the given answer does not actually use any of those documents. For that reason, while it is helpful for clarifying the context, the source list should not be seen as accurate representation of the claim-source relationship.

5.9 WEB FALLBACK

tavily_search looks for a missing key and gives an empty list if web search is not possible. If there is a key, it can ask for maximum five search results with minimum search depth. The result will be a Document with fields title, URL, and response where is_web is set to be true in the metadata.

The graph substituted local documents by these web documents. If the web documents are available, the generation takes place. The source list contains web URLs, and the graphed text offers an alternative to the warning message by providing the explanation. The explanation is good about provenance but hides the actual warning created by the generator, like any kind of troubleshooting.

The better design would keep both route notification and risk analysis separately. It would also check web evidence against the initial information need and, if necessary, use domain of the source and relevance rules.

5.10 CONVERSATION STORAGE

The script called database.py sets up SQLAlchemy, making sure there is a connection, and a timeout of 5 seconds if the connection fails; it also creates tables if the connection is good. If the connection does not succeed, the script creates a flag set to disconnected and later pushes a False response.

The function save_conversation_turn creates a parent conversation if needed, uses the original question to make a title, saves the turn information, and commits the transaction; it rolls back if there is any issue. However, the user of the function does not get information about the returned success status value. Thus, users can see what looks like a success even if there is no successful saving of the information.

The function get_conversation_history first gets the list of the conversation turns, orders them, and applies the limit equal to 50. The description of the function states that it gets a history of the latest turns from the conversation, which is misleading, as it gets to the first matching turns.

This way, users can be misled when using long conversations. In order to fix the issues, the function should obtain new matching records and only then reorder them before displaying.

5.11 IMPLEMENTATION BOUNDARIES

The analysed repository does not provide model-training code, any evaluator's weights, benchmark results, or application test suite. The contribution of this resource is constituted by functionality realization of services and control process.

It is pertinent mention that this separation is crucial since the production-grade description provided by the README does not mean you have performed actual deployment of the resource.

There is also another issue, which is the compatibility of annotation. For instance, the use of Optional[Any] in an upload function in ingestion.py was not preceded by importing Any module. Under the eager function-annotation evaluation included in Python 3.12, such definition will raise NameError exception. Such behaviour was confirmed through checking definition in isolation. However, the result is only the proof that the issue happens in this particular version and does not result in the same behaviour in other versions with different semantics of annotations.

As to the excerpts found in Appendix 4, it should be pointed out that they are being provided fully marked as taken from the repository and revealing the mechanisms of control and retrieval. They are presented only for the technical evaluation and should not be considered the whole clone of the tree.

Chapter 6
TESTING AND EVALUATION METHODOLOGY

6.1 EVIDENCE LEVELS

In this report, three types of data have been identified. The source inspection method shows what functions and programs have been recorded in the code. The method of isolated offline execution is able to verify the working of certain functions without any means of control. The method of a live integration test helps to understand the way a set of services acts in certain conditions. Only the first two methods have been implemented in this study.

Using these methods of differentiation reduces the possibility of a typical error in reporting: the function-level test can show successes or failures of executing functions without knowing whether the whole is functioning properly. Also, the schema that gives a confidence value cannot determine the correctness of a value. Thus, each result obtained is valuable and has its limits.

No modifications were made to the analyzed source code that would result in the reported results. A task-local harness processes the code using the abstract syntax tree module of Python and extracts necessary definitions from the source. Small substitutes are capable of providing document objects and type names when necessary. The original processed functions are being executed without changes.

6.2 OFFLINE CHECK DESIGN

The first inspection involves checking all 13 Python files. This ensures that syntax is correct using the review interpreter without interaction with any services. The checking does not require the importing of all modules or resolving version dependencies. While the syntax is correct, it is entirely possible for the file to fail, when either a function or an import is checked later.

Retrieval checks do not need large amounts of labelled passages, they are trying to check the promotion of passages which are present in both rank lists, result cap, handling of cases with empty input, deduplication, and recognition of different sources. The cases here are primarily trying to verify the visible ranking contracts instead of making sure that the function returns a list.

The checks for source formatting ensure that the exact page is retained, and that there is stable deduplication and no creation of new pages. Vector metadata checks ensure that any metadata

values that may be unsupported are converted and chunk identifiers are stable. It also checks for routing by looking at greeting bypass, the wide personal-introduction prefix, handling of the web result and routing with insufficient evidence when there is no document available on the web.

The annotational verification retrieves the original definition of ingest_uploaded_file and its corresponding notation, as well as all the other referenced type names. While performing analysis within Python 3.12, definition evaluation fails with NameError regarding the type Any. This enables detection of compatibility issues without falsely claiming that the whole application was launched.

6.3 EXECUTION ENVIRONMENT

The evaluation was performed on the operating system known as Windows that comes with the integrated Python 3.12 runtime. At the outset, due to the absence of an HTTPS helper by an already available Git executable, Git history was cloned instead. The commit hash used in Chapter 1 indicates the precise source material to be assessed and will not point to an elastic branch name.

No inference keys were made during the assessment. Since the evaluation was carried out offline, no Pinecone index was created nor was the document transformed into an embedding through Cohere, or questions sent to Gemini, or queries made through Tavily, or PostgreSQL tables made available for checks, and thus everything was made without considering service availability, although the conducted assessment is indeed much narrower than the integration testing.

For analyzing the provided PDF, the corpus artifact approach was employed. The text extraction procedure allowed establishing what is referred to as the title and content extent, as well as the overall structure. Thus, one cannot claim that the chatbot correctly addressed the list of medical questions because there is no particular list of questions to be tested and approved by a human expert.

6.4 PROPOSED QUESTION SET

The future assessment should start with a predetermined set of questions and these questions should be used in checking the performance of the system instead of looking for examples based on the responses of the system. The intended question set should have direct questions requiring the extraction of facts, paraphrasing tasks, questions involving acronyms, questions

requiring comparison of texts, questions to be asked in dialogues, impossible questions, and questions requiring web responses only.

An example of a designed question set includes 100 questions: 30 questions that require direct factual information to be looked up, 20 paraphrased questions or terminological variants, 15 questions requiring many texts for comparison, 10 conversational questions, 15 questions that cannot be found in the current corpus of the place, and moreover, 10 messages with greetings or questions concerning personal life. This is only a proposal about the distribution of questions and does not mean that the data has been collected and tested.

In case of each question an annotator is expected to take down information about evidence boundaries, acceptable elements in the answer, relevant page and chunk identifiers, as well as whether abstention is appropriate. Then the dispute should be reviewed by a second annotator. Ambiguous questions must be clarified or stated explicitly, since otherwise disagreements about the question may be interpreted as a failure of the model.

6.5 BASELINES AND ABLATIONS

To describe the simplest baseline, we first retrieve dense retrieval and generation without any correction loop. A second baseline uses only BM25. A third baseline employs one-pass hybrid retrieval and uses the same generator and context budget. This full system then incorporates verification, bounded rewriting, and web fallback. By fixing both the generator and the corpus, we can interpret the comparison.

Such ablation experiments will examine change by removing one decision at a time. Removal of query optimization shall test to see whether rewriting helps before the first search. Turning off the corrective rewrites will allow testing their benefit after weak evidence. Turning off web fallback will isolate the performance on the local corpus. Adding in verification will test the impact of fixing the run-time gap in control flow.

Comparison should not quietly change provider models or the document snapshot across runs. It should record the number of chunks selected and their total context length. Otherwise, any apparent gain due to the new decision rule may actually come about from a wider evidence budget or a change in quite different models.

Table 6.1 Proposed comparison configurations

ID	Retrieval policy	Purpose
B1	Dense only and one generation	Dense baseline

ID	Retrieval policy	Purpose
B2	BM25 only and one generation	Lexical baseline
B3	One-pass RRF hybrid	Value of fusion
B4	Hybrid plus evidence verification	Value of a sufficiency gate
B5	Verification plus bounded rewriting	Value of local correction
B6	Current full graph with web fallback	Observed implementation policy
B7	Full graph with verified web evidence	Proposed repair comparison

6.6 MEASURES

Retrieval recall at k refers to the concept of determining if the acquired chunks collect annotations relevant proof and the concept of precision at k implies measuring how much of the gained content can be marked as relevant. In case of problems with several facts being needed, evaluating the coverage should be done at the level of facts or evidence spans rather than only judging one of the chunks as valid.

When assessing answers, it is necessary to distinguish between correctness, completeness, and support. Correctness signifies whether an answer corresponds to the reality whereas completeness indicates if the question has been answered in its entirety. As to support, this criterion is used for evaluating the extent to which an answer can be drawn from the documents that were provided.

Abstention must be evaluated in both directions; answering an unanswerable question is a mistake but refusing to answer an answerable question is also a mistake. A valid report should provide the rate of unanswerable questions which are, however, refused correctly and the rate of answerable questions which are, however, refused incorrectly.

Latency must be measured from the question submission to the final answer provided, before the display animation made by the UI. The median and a high percentile must be included in the years of retrieval requests and data provided over the years. For cost, the actual input/output

and expenses made by the providers should be recorded instead of adopting undisclosed estimates.

6.7 CONFIDENCE AND CALIBRATION

The generator provides a level of confidence, which then becomes visible on the interface as an evidence grounding. The intended scope of calibration can be evaluated by dividing the answers into confidence levels and checking them against their independently evaluated correctness or proof. The event for which the needed level of confidence is evaluated must be clearly defined: level of confidence in factual correctness cannot be viewed as synonymous with the level of confidence in the fact that it has been supported by the retrieved evidence.

Fixed values coming from direct chat and refusal threads must be analyzed separately from the values produced by the generator. If these values were grouped in the same chart, they would have conflicting meanings. The direct chat value of 1.0 (received with the help of a code) is different from the value of refusal of 0.1 (also received with the help of the code).

The first step to improvement would be to rename the displayed value so as to explain its origin and avoid misleading the user into believing it is something validated.

6.8 FAILURE AND RESILIENCE TESTS

For integration testing, the service must be deliberately made to fail one at a time. In this regard, one should test the embedding service that is not available during the ingestion phase. Besides, the integration test can check if the current index is not compatible, the vector response is empty, the input structured model is not well-formed, the keys on the web are missing, and the write to the database does not function properly.

Testing documents should include image only pdf’s, the malformed pdf’s, the slideshow with shaped shapes, the slideshow that has notes, the old ppt, the text that is not compatible, and the modified document that already has some previously indexed parts. The cases are very useful in checking if the library display and searchable corpus are appropriately working.

The last level of testing is the security one. In this testing session, path correctness must be checked, filenames inserted into HTML must be properly escaped, the access to documents must be shared between different users, and instructions placed inside the retrieved document must be treated properly. These tests can be included in the proposed tests; however, there was no full security testing of the system.

Chapter 7
RESULTS AND DISCUSSION

7.1 RESULTS OVERVIEW

Seventeen assertions undertaken via offline channels were accomplished successfully. This implies that assertion matched the expected output for a source in question. However, this does not mean seventeen product capabilities conform to the correctness of particular features, as some assertions have been tested intentionally to confirm a defect or limitation. The fact that all thirteen python files were syntaxially parseable is worthwhile mentioning, as well as the fact that isolation check of the annotations has produced a runtime definition error under Python 3.12.

The most notable positive findings have to do with deterministic logic. RRF facilitates the agreement of lists, maintains separate provenance and limits the number of results. Formatting of the sources preserves the existing pages and deduplicates identical citations. The identifier derived from the content remains stable for the unchanged fragment of the text.

The most significant negative finding has to do with control and consistency. A long conversational prefix helps bypass retrieval. Web documents are accepted for generation without the assessment of sufficiency. The modified content of the document produces a new vector id but does not remove the old records.

Table 7.1 Executed check groups

Check group	Count	Observed outcome
Python syntax	1	All 13 source files parse
RRF behaviour	5	Agreement, cap, empty input and identity rules confirmed
Source formatting	2	Deduplication and missing-page handling confirmed
Metadata and chunk IDs	3	Cleaning, repeat IDs and changed-content IDs confirmed
Conversational routing	2	Greeting bypass and broad-prefix defect reproduced
Annotation compatibility	2	Missing import identified and NameError reproduced

Check group	Count	Observed outcome
Web routing	2	Nonempty results generate; empty results abstain
Total	17	All assertions match expected source behaviour

7.2 RETRIEVAL FINDINGS

The controlled RRF example illustrates that agreement between sparse and dense lists can be more significant than high ranks on either lists, which is the purpose of the method in use. The results also show that passages with the same text but different metadata do not merge, which needs to be remembered when it is important to ensure the correct source identity.

The tests do not indicate that the resulting order of the documents is suitable for the WHO report or any other corpus, as it relies on synthetic documents being labelled in order to study ranking. The assessment of relevance presupposes real questions and evidence being pinpointed independently.

The method uses dense similarity metrics before producing the final ranking. As this method uses ranking based on similarity scores, it does not mean that a list cannot contain irrelevant documents. Therefore, the verifier plays an important role in the system and its false positive rate should be applied in the future experiments.

7.3 ROUTING FINDINGS

The greetings test validates the finding that a specific common greeting yields a False outcome without a model invocation. The broad prefix test confirms that a sentence that starts with the words "I am" can lead to identical results although the message is related to documents. This is a shortcoming in the definition of the rule, not an unexpected conclusion of the language model.

To fix this problem, the way the introduction rule is defined should be limited so that the mixed intent communication is being tested. For instance, observe the greeting followed by information question, the introduction followed by document request, or expression of interest followed by comparison. Therefore, the router will need to assess complete information needs and not just the first phrase.

The web route test yields the same clear conclusion. A non-empty document list goes to generate_answer in spite of enough_evidence being False. Empty list goes to

handle_insufficient_evidence. This way the branch decision is confirmed in a standalone manner, and the architecture diagram directly supports the edge between web search and generation.

7.4 EVIDENCE CONSISTENCY

It can be seen that the uncontrolled branch of web leads to inconsistency in the final state of webformed answer. This indeed can be produced with web snippets even when enough_evidence is False, and verification_reason shows that local evidence could not be verified. As a result, anyone who reads the stored turn might assume that the flag means something about the web answer instead of understanding that it is not the case with those documents.

The suggested way of improvement is to limit evidence conclusions to the context they are referred to. There are two possible strategies. First, the verification can be repeated right after the web search with the changes of all related fields being made. Second, local and web verification could be conducted independently. In any case, the answer should use the same evidence set that was involved in generation process.

The issue of warning replacement is another inconsistency issue. In the presence of web metainformation the actual warning of the generator is replaced with a different one. By means of provenance notice one should not remove any generation mistakes and claim concerns. The introduction of separate route_notice and risk_warning fields should solve the.

7.5 INDEX CONSISTENCY

The startup index setting simply indicates

if a remote index has vectors. However, it does not check the vectors against the current local documents. BM25 is based on the information in the local repository. As a result, it can create hybrid retrieval of different corpus states after adding documents manually or changing existing files.

Stable content ID avoids inserting the same parts more than once but does not help with deleting older parts. The document manifest should contain a file hash, parser version, chunking settings, embedding model, and IDs of the version of a document. With a new file uploaded, therefore, we can deactivate or delete old records.

Until a similar mechanism is created, we should take the Refresh Index button not as a full remote rebuild. At this moment, in the current UI, it clears all cached data and performs startup.

Pinecone index that is not empty already can still be used without adding the local files manually.

7.6 COMPATIBILITY AND REPRODUCIBILITY

The annotation failure constitutes a direct finding of compatibility. Ingestion.py refers to the type Any without importing it. The isolated definition fails in Python 3.12 despite passing syntax parsing. As pipeline/__init__.py imports ingestion, the issue can influence imports at the entry point.

The README mentions testing on Python 3.14 but it does not include any execution records to back that up. Different semantics in annotation create a reason to reject generalising the result for Python 3.12 across the runtimes. The only reliable conclusion is the need to have an explicit import of Any in the source itself.

The list of dependencies mentions minimum versions being used but does not have a lock file. The README mentions smoke-test module and license file but those are not present in the snapshot. Those discrepancies should be resolved before showing that the setup is reproducible. Having a versioned environment and a small integration suite would serve as a more solid proof than documentation claims.

7.7 REQUIREMENT ASSESSMENT

The repository has mechanisms in place such as ingestion, hybrid search, local evidence retrieval, bounded correction, optional fallback to web search, two interface types and optional persistence. Source analysis confirms that the repository supports all the mechanisms described above but does not guarantee their successful functioning in a real-world deployment.

Traceability is imperfect: some identifiers are retained, but there is no information about specific claims or their complete tracking histories. Partially implemented bounded local correction does not contain the stated end-to-end real-time budget. The persistence option may not work without being notified. Multiple-user isolation could not be demonstrated due to the use of default name space and folder structure of the data set.

The project can be characterized as a modular RAG prototype with working control logic. Its design allows for further development of the prototype and testing of its other parameters. However, in order to claim that the project is ready for practical implementation, additional evidence of the reliability of integration processes, access management, corpus life cycle tracking, monitoring and answer quality should be obtained.

7.8 THREATS TO VALIDITY

The offline harness relies on limited substitutes for framework-related entities. It inspects selected function bodies but does not take into account all interactions with LangChain or any instance of LangGraph, Pydantic, or provider SDKs. Thus, it does not find all conflicts among dependencies and formats used by various services.

The source study pertains to only one commit. Later amendments may remedy or create the new behaviour while the APIs of providers may evolve irrespective of the code. The findings of the research should be verified based on the relevant snapshot before looking for the differences with the other versions.

Various studies represent different methods of evaluation, data use, and corpus use. The improvements introduced in one study cannot be converted to the number applied to this paper. Similarly, having large PDF does not imply that it can be generalized outside the type of the document. A larger research project should deal with a number of different document collections having different forms and vocabularies.

7.9 INTERPRETATION

The results demonstrate that an engineering lesson can unequivocally be drawn: a correction loop can trust its evidence and state. Introducing retries alone does not help when the evidence is old or the alternative branch omits a check that served as the basis for designing it. At the same time, minimal deterministic elements like source formatting and stable IDs can contribute to the success of design solutions if one can adequately explain what guarantees are provided by such elements.

Future evaluations need to inherit this logic. First, fixing import issues is necessary to align dense and sparse evidence. After that, the web branch must be checked so that warning semantics can be kept. Only after these two steps can the effect of corrections on support and completeness be evaluated..

7.10 OBSERVABILITY AND MONITORING RESULTS

The next illustrations reveal the measures of observability and monitoring acquired from LangSmith during system testing at October 15, 2026. The metrics give details about the number of traces, the delays in the workflow, the calling of LLM, the number of tokens used in the processing, and the cost assessment.



Figure 7.1: Trace Count – Total number of successful and error traces over time

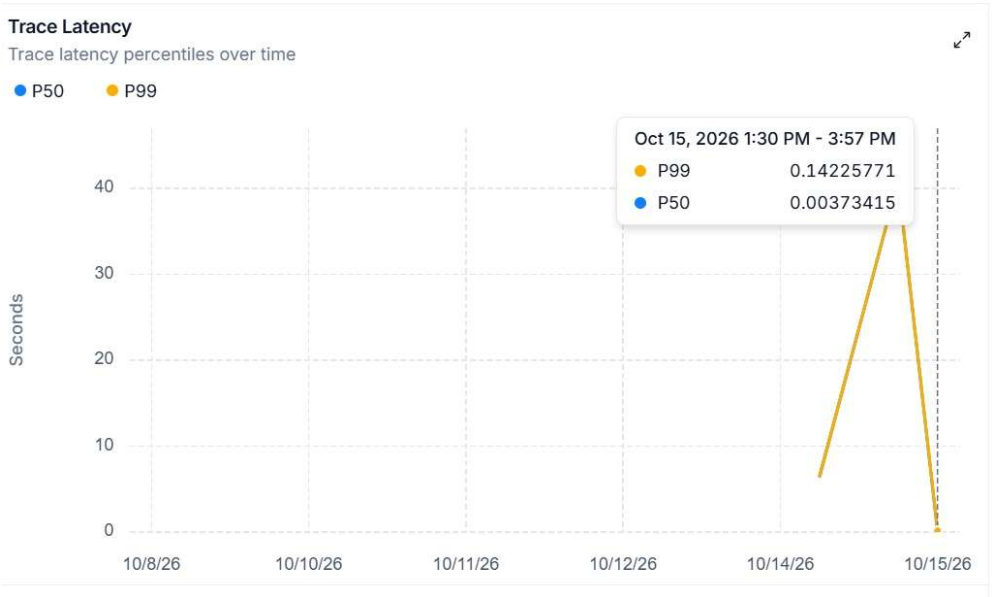


Figure 7.2: Trace Latency – P50 and P99 latency percentiles over time

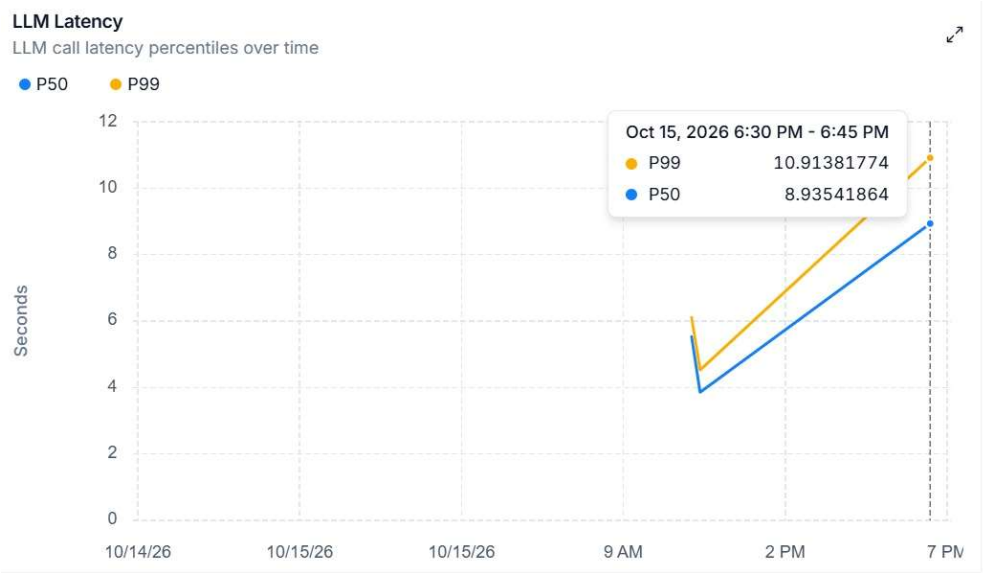


Figure 7.3: LLM Latency – P50 and P99 LLM call latency percentiles over time

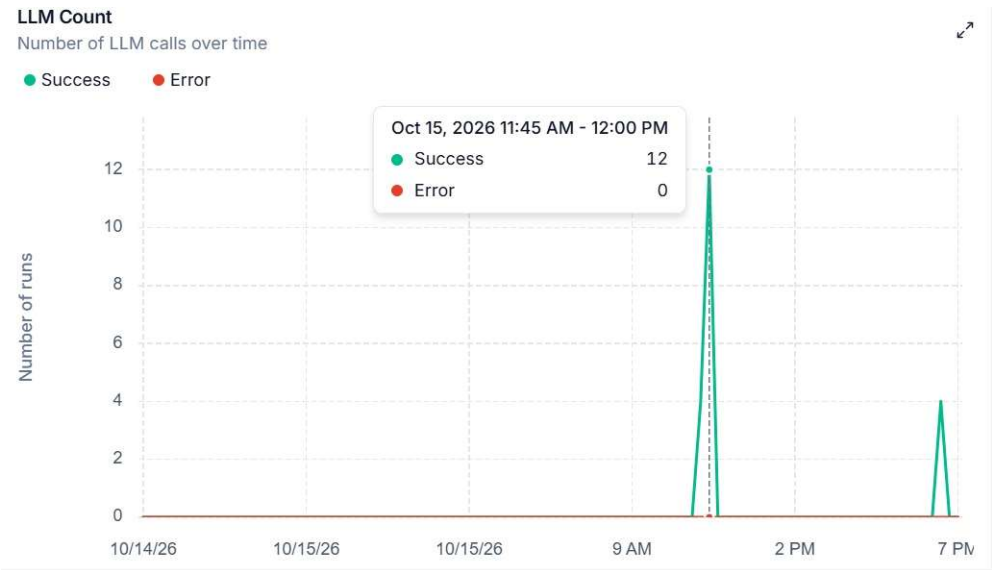


Figure 7.4: LLM Count – Number of successful and error LLM calls over time

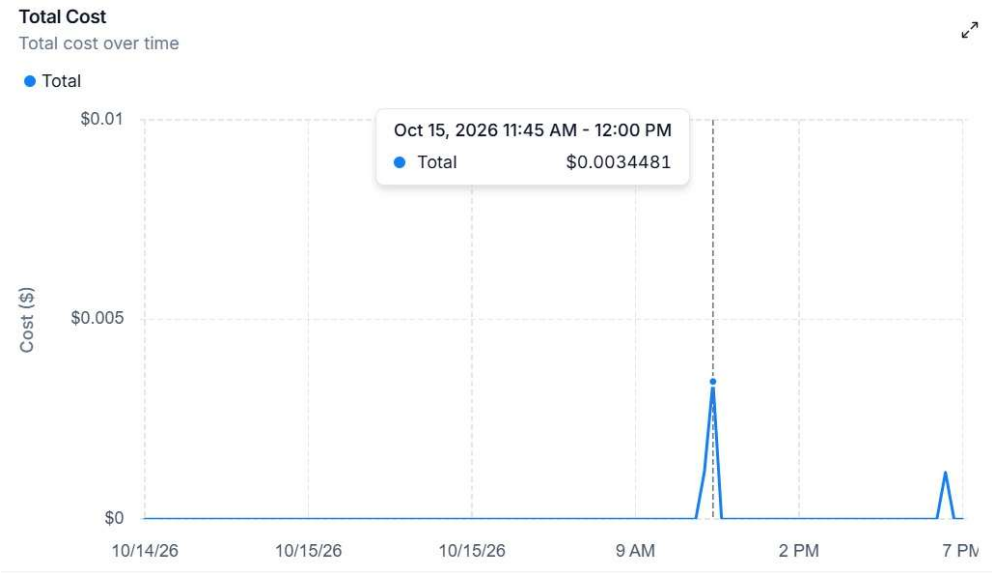


Figure 7.5: Total Cost – Cumulative API cost over time

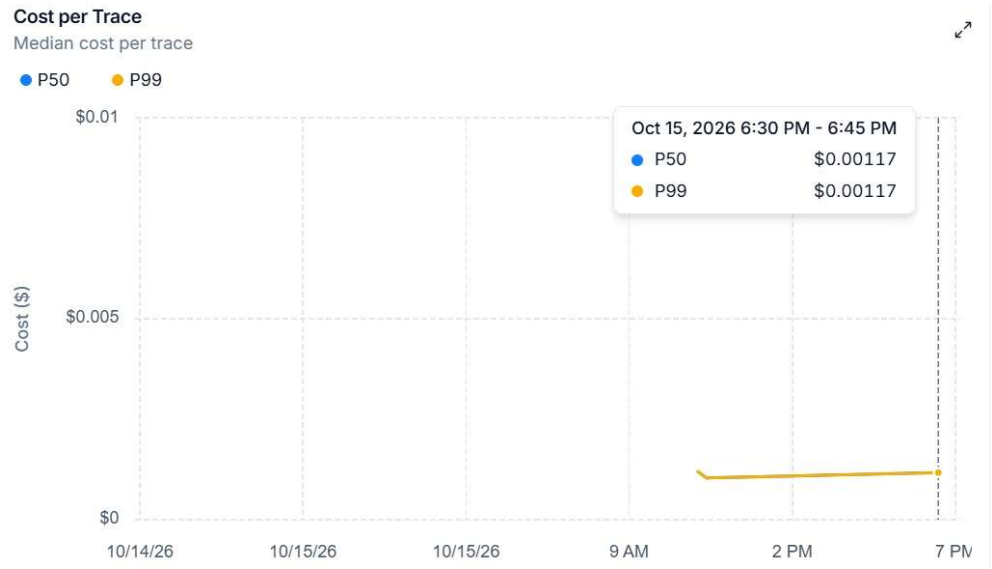


Figure 7.6: Cost per Trace – Median cost per trace (P50 and P99) over time

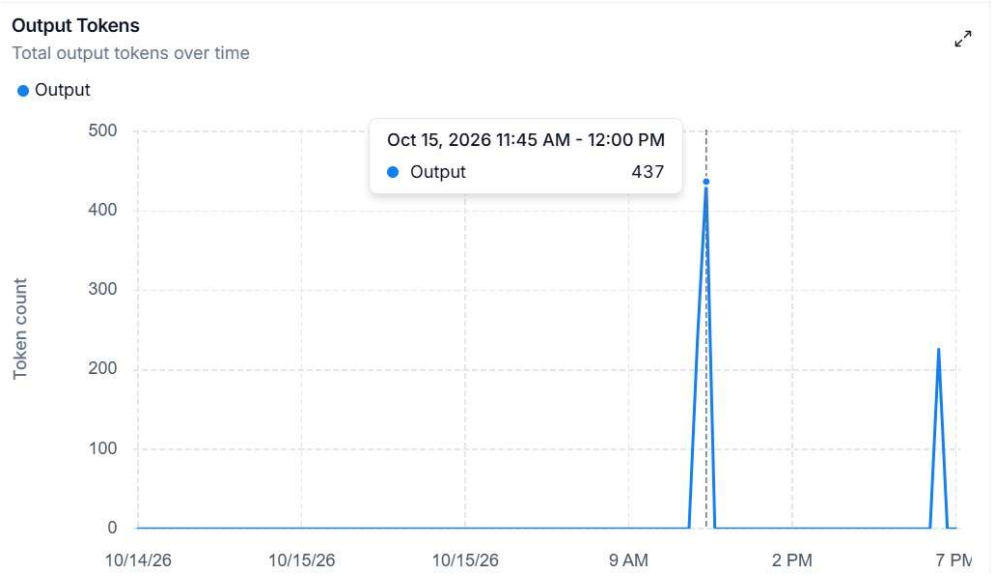


Figure 7.7: Output Tokens – Total output token count over time

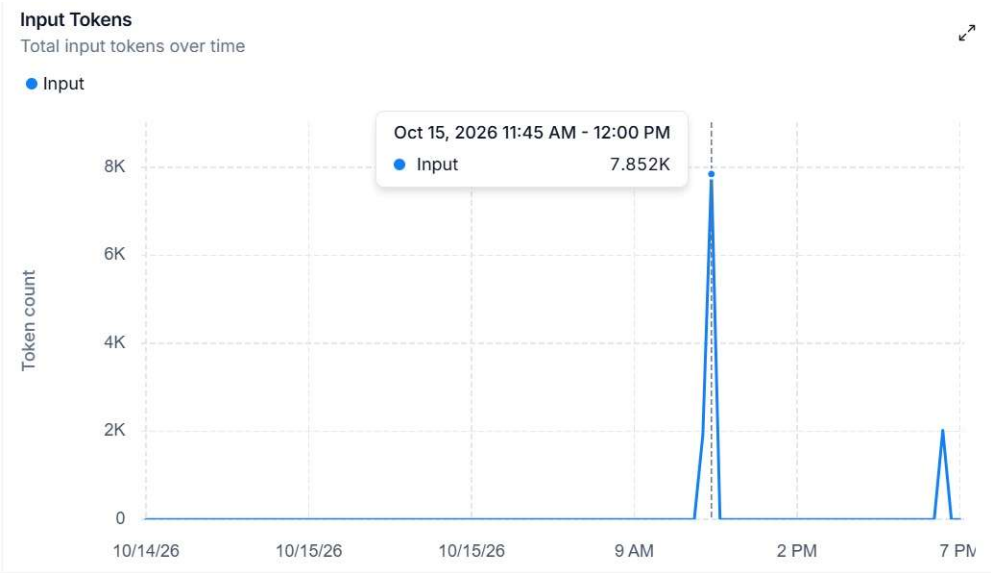


Figure 7.8: Input Tokens – Total input token count over time

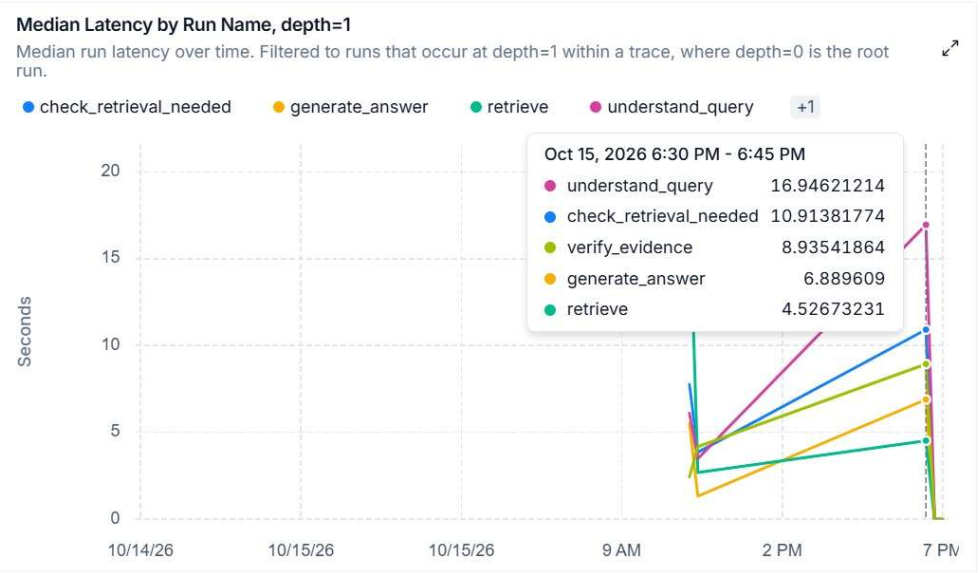


Figure 7.9: Median Latency by Run Name – Per-component latency breakdown at depth=1

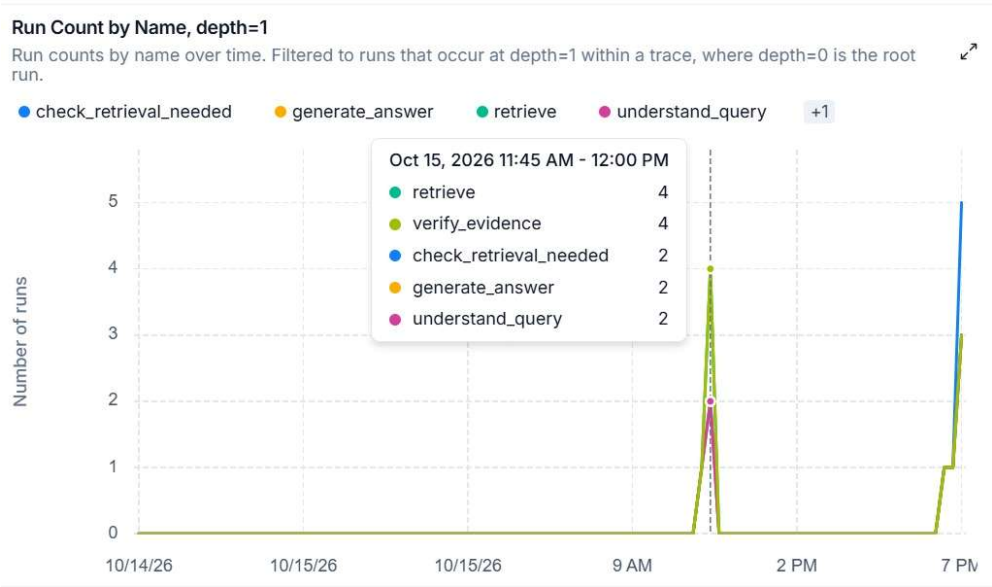


Figure 7.10: Run Count by Name – Per-component invocation counts at depth=1

Chapter 8
CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

Adaptive CRAG offers a competent retriever functioning within an affordable compact application. The tool implements dense and keyword retrieval, leverages language model support for checking local evidence, formulates unsuccessful queries, and resorts to web search when the local collection does not provide enough resources.

The source evaluation and offline verification demonstrated a number of significant features. Rank fusion works well with controlled lists. The source formatting preserves the practically traceable origin of information while avoiding the issue of page numbers. The chunk ID is guaranteed to remain the same for the unchanged content. The local attempt counter has clear limits.

The same analysis becomes a critical limitation that has to be taken into account for the goals of the project. Outcomes of the web search deviate from the verifier, the degree of the confidence was not calculated, lists of sources do not consist of claims making references to the context, and locations of the local articles and articles retrieved from the source might vary. The usage of a random routing prefix may exclude the process of retrieval for substantial inquiries. Python 3.12 has revealed an annotation name that remains unclear in the module responsible for gaining information.

The concept is an appropriate basis for further improvement. The possible improvement regarding the quality of the inquiry requires further examination as it has not been evaluated yet, and the flaws discovered should be fixed before launching the solution.

8.2 IMMEDIATE IMPROVEMENTS

The first enhancement is a consistent startup process. When Importing Any, one should do a dependency check, establishing the environment version the code was developed in, and checking the readme information against config.py. The idea is to create a simple import and configuration check. This allows bringing down the chances of getting false results due to the environment differences.

The second enhancement is consistency in evidence. Contextualize the web results, keep the verification results attached to the context of the web results, and split route messages from the

risk messages. Actually verify the standalone query after retrieval and before generation stage so that the context remains unchanged throughout all the dialog turns.

Thirdly, corpus lifecycle management should be properly performed. This involves maintaining a list of all document versions along with corresponding chunk IDs, deleting old vectors, or matching the index refresh with the label of the operation. In summary, dense and sparse retrieval processes have to work on the same approved corpus version.

8.3 EVALUATION PRIORITIES

When all modifications are completed, construct the specified set of questions and evaluate various ways of addressing them, including dense, sparse, hybrid, and corrected ones. Take note of meanings and answers that come up during processing. Answer support and abstention as well as correctness ought to be assessed too. It is crucial that more questions answered do not equal better scores if the related responses lack proof.

Confidence has to be judged separately from answer quality. It is necessary to interpret the displayed score properly and provide results of the calibration study. Citation evaluation should check if each fact has been proved by the quote rather than just be praised for a long list of sources.

Latency and service use must be measured along with quality of answer. Per-question traces should prove which retries are good for retrieving supporting evidence and which just cause a delay in getting the answer.

8.4 LONGER TERM EXTENSIONS

In the updated edition of the system, support for citations associated with claims, focused context use, a re-ranking function, and an improved router for multi-question problems may potentially be introduced. The improvements should be employed one at a time so that changes can be measured. Due to the modular representation of the components, it is possible to make adjustments in a controlled manner, although with each model application, there is an additional chance for failure.

Authentication, as well as document-level access management, will be necessary for the pilot model created for shared usage. Retention rules, deletion action, limitations for upload, and health monitoring will also be required. The aim is to ensure the reliability of the boundaries of evidence for every customer regardless of the fact that chat messages are visually improved.

REFERENCES

1. Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). *Self-RAG: Learning to retrieve, generate, and critique through self-reflection.* arXiv. https://doi.org/10.48550/arXiv.2310.11511
2. Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758–759). https://doi.org/10.1145/1571941.1572114
3. Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 150–158). https://doi.org/10.18653/v1/2024.eacl-demo.16
4. Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 6465–6488). https://doi.org/10.18653/v1/2023.emnlp-main.398
5. Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.-W. (2020). Retrieval augmented language model pre-training. *Proceedings of the 37th International Conference on Machine Learning, 119* (pp. 3929–3938). https://proceedings.mlr.press/v119/guu20a.html
6. Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume* (pp. 874–880). https://doi.org/10.18653/v1/2021.eacl-main.74
7. Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1* (pp. 7036–7050). https://doi.org/10.18653/v1/2024.naacl-long.389

8. Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 6769–6781). https://doi.org/10.18653/v1/2020.emnlp-main.550
9. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474. https://arxiv.org/abs/2005.11401
10. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157–173. https://doi.org/10.1162/tacl_a_00638
11. Ma, X., Gong, Y., He, P., Zhao, H., & Duan, N. (2023). Query rewriting for retrieval-augmented large language models. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 5303–5315). https://doi.org/10.18653/v1/2023.emnlp-main.322
12. safal11goyal. (n.d.). *CRAG* [Source code, commit 1cd0468ea67a24127b3692aa9c6d192d5e2eae97]. GitHub. Retrieved October 12, 2026, from https://github.com/safal11goyal/CRAG
13. World Health Organization. (2025). *Global tuberculosis report 2025.* ISBN 978-92-4-011692-4. Corpus copy: https://github.com/safal11goyal/CRAG/blob/main/documents/TB_2025.pdf
14. Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). *Corrective retrieval augmented generation.* arXiv. https://doi.org/10.48550/arXiv.2401.15884

Appendix 1
REPOSITORY INVENTORY AND OPERATING PROCEDURE

A1.1 SOURCE PROVENANCE

This report discusses the public repository of https://github.com/safal11goyal/CRAG in commit 1cd0468ea67a24127b3692aa9c6d192d5e2eae97. The snapshot was made on 12 October 2026. The ownership of the repository and authorship of the code is unchanged from this source. Explanatory analysis, diagrams, literature syntheses and offline reviews for this report were prepared with that regards. The README file could suggest that there is an MIT licence, while no LICENSE file is present in the repositories reviewed. This report notes about this inconsistency, but does not mention how to get the missing licence terms. The parts highlighted in the chosen listings represent specific technical parts of the project; the licence issues of the repository are to be settled by the developer when distributing the derived software product

Table A1.1 Complete application file inventory

File or group	Role
app.py	Streamlit interface and pipeline initialisation
main.py	Terminal interface and command loop
config.py	Typed settings and configuration validation
database.py	ORM models and persistence operations
pipeline/__init__.py	Package exports
pipeline/ingestion.py	Loading, chunking and upload ingestion
pipeline/pinecone_store.py	Vector index wrapper
pipeline/retrieval.py	BM25 and reciprocal rank fusion
pipeline/query.py	Routing and query reformulation
pipeline/verification.py	Structured local evidence assessment
pipeline/generation.py	Answer output and source formatting
pipeline/web_search.py	Tavily result conversion
pipeline/graph.py	State and corrective graph

File or group	Role
requirements.txt	Minimum dependency declarations
README.md	Project description and setup instructions
.gitignore	Ignored secrets, caches and generated data
.streamlit/config.toml	Theme and server settings
documents/TB_2025.pdf	76-page example corpus document

A1.2 PREPARATION

Make sure to set up an isolated environment in Python and to install all the requirements of the project. Before carrying out the operations make sure to resolve the missing import of any functions that require interpretation of the annotation. Check if the Docker package is installed. After that make sure that you have noted down the required versions of the interpreter and the packages. The above-mentioned procedure.

Create a new .env file in the project's directory that contains Gemini, Cohere, and Pinecone credentials. Tavily should be configured only if external web evidence is planned to be used. PostgreSQL deserves a configuration only in case if durable history is required. Keep all the sensitive information outside the native version control system. Never copy and paste any keys to the background screenshots, logs, or reports.

Check whether the required identifiers for the model are consistent with the accounts that were used in the process of the experiment. Use a separate index or a namespace for data processing purposes. Always vary the embedding model and the index parameters before duplicating the data..

A1.3 RUNNING THE INTERFACES

The streaming interface runs the command streamlit run app.py in the directory of the repository. The command-line interface is launched with python main.py. Documents for local processing should be either saved into the action folder, or uploaded via the web interface. The command-line tool verifies that the action folder nonempty before starting to initialize the pipeline.

In order to show how the system works, one can ask the supported factual question, one paraphrase question, one question referring to greeting, and one author question. Pay attention

to the different source labels, alerts wrote down the used route. Save the original question and the final checks..

A1.4 REPRODUCIBLE EXPERIMENT RECORD

Retain the source commit, the versions of the environment, model identifiers, hashes of the corpus, chunking methods used, number of retrievals carried out, maximum retries, question pool version, and whether web search is enabled or not for each run. Using timestamps around the graph execution, log the start time and the end time. Keep the traces of intermediate queries as well as the outputs of all runs.

Make sure to document configuration changes while making comparisons; for instance, changing the generator together with changing the chunk size and using the reranker does not allow to identify the source of any improvement..

Appendix 2
OFFLINE CHECK RECORD AND PROPOSED ACCEPTANCE TESTS

A2.1 EXECUTED ASSERTIONS

The report that follows lists every check that was carried out in the review. PASS indicates that the behavior exhibited in the test conformed to the assertion. Also, these reports are not representative of quality and integration measurements of external services.

Table A2.1 Executed offline assertion record

ID and result	Check	Observed evidence
T01 PASS	Python syntax	13 Python files parsed
T02 PASS	RRF shared result promoted	Beta ranks first when present in both lists
T03 PASS	RRF cap	Output limited to two
T04 PASS	RRF empty inputs	Empty result preserved
T05 PASS	RRF deduplication	Whitespace-normalized same source and page merged
T06 PASS	RRF provenance distinction	Identical text from different sources retained
T07 PASS	Citation deduplication	Stable order and exact page
T08 PASS	Citation missing page	No invented page
T09 PASS	Metadata cleaning	Null omitted and numeric list stringified
T10 PASS	Stable chunk ID	Fallback index does not change content hash
T11 PASS	Changed content ID	Content edit creates another ID
T12 PASS	Greeting bypass	No model required for greeting
T13 PASS	Broad prefix bypass	Factual intent can be swallowed by i am prefix
T14 PASS	Missing Any import identified	Any annotation exists but typing import omits Any

ID and result	Check	Observed evidence
T15 PASS	Python 3.12 annotation failure	Unchanged function definition raises NameError for Any under this interpreter
T16 PASS	Nonempty web bypass	Documents alone trigger generation
T17 PASS	Empty web abstention	No web results route to refusal

A2.2 PROPOSED ACCEPTANCE SCENARIOS

The cases below will be used in the future connected test environment. They outline certain behaviors to assess but do not make any finality claims. The evidence questions need to be instantiated with the extracts and facts that a human annotator will check.

Table A2.2 Proposed acceptance scenarios

Case	Input or condition	Acceptance evidence
AT1	Question with one clear source span	Correct answer and inspectable supporting page
AT2	Paraphrase with different vocabulary	Relevant evidence retained by retrieval
AT3	Question absent from local corpus	No unsupported local answer
AT4	Insufficient local evidence and useful web result	Web context verified and route disclosed
AT5	No local evidence and unavailable web search	Explicit insufficient-evidence response
AT6	Greeting followed by a factual question	Factual request is not bypassed
AT7	Follow-up containing a pronoun	Same resolved intent reaches all factual stages
AT8	Updated file with changed text	Old chunks retired and both indexes agree
AT9	Database write failure	Answer retained locally and persistence failure visible
AT10	Two users with private documents	No cross-user retrieval or history exposure

Case	Input or condition	Acceptance evidence
AT11	Image-only PDF	Extraction limitation reported clearly
AT12	Legacy binary presentation	Conversion request instead of false indexing success

A2.3 ACCEPTANCE INTERPRETATION

Accuracy of a response is determined through validated response components and content backing the response. Just believable answers cannot constitute valid sources of evidence. If a response is not present, a simple no should be used. No should count as a coverage error in case of presence of a corresponding answer.

There is no threshold number to be accepted without the implementation and tagged pilot set. University demonstrations allow slower responses than interactive customer support while critical information service needs stricter measurement of evidence. One should set thresholds prior to executing the actual trial and not change them according to the results already obtained.

Appendix 3
ALGORITHMS AND COST ACCOUNTING

A3.1 CORRECTIVE CONTROL ALGORITHM

The two stages are applying regex on the inquiry to verify whether the inquiry matches with any message in the dialog history. If a match occurs in this case, the direct-chat approach creates the response based on the dialogue matching the inquiry.

In case there is no match or the dialog history is not available, the search query is created. Also, each attempt retrieves evidence, increases the number of attempts made and matches the retrieved evidence against the inquiry. Once the evidence is adequate, the generator receives the context. But in case there is not enough evidence, the user rephrases the query.

```
question, history = input
if not retrieval_needed(question, history):
   return direct_chat(question, history)
query = understand_query(question, history)
attempts = 0
while True:
   documents = hybrid_retrieval(query)
   attempts += 1
   verdict = verify(question, documents)
   if verdict.enough_evidence:
      return generate(question, documents)
   if attempts >= maximum_attempts:
      break
   query = improve(question, query, verdict.reason)
web_documents = search_web(query)
if web_documents:
   return generate(question, web_documents)
return insufficient_evidence_response()
```

A3.2 REQUEST ACCOUNTING

In order to answer a fact-based question through m local efforts, it is assumed that the router was used only one time without errors and retries from the provider. The first step of initial understanding makes use of one model call, verification requires m calls, rewriting utilizes m-1 calls, and the generation of information involves one model call. Given these assumptions, the maximum value equals 2m+2.

The default limit makes the web-answer model similar with a total equal to 8 model calls under some assumptions plus one web-search request. In a case of no local retrieval, a call by the

verifier is disabled, so the number may vary in reality. Rate of retries increases the number of requests for the connection, thus making the previous equations an estimate only.

Any dense retrieval implies a query that initiates a vector search. The sparse retrieval takes place locally. The second phase involves separate requests for document embedding and vector upsert and dimensionality-probing. Therefore, the total activity will be underestimated by the evaluations of service activity..

A3.3 RECORDING EXPERIMENTS

A useful per-attempt record contains question_id, attempt_number, original_question, search_query, dense_chunk_ids, sparse_chunk_ids, fused_chunk_ids, verification_decision, verification_reason, and elapsed_time. The final response record should add generated_answer, claim_citations, route_notice, confidence, model identifiers, and service usage when available.

These fields are proposed for evaluation logging. The existing conversation_turns table stores only a subset and does not retain the ranked candidate lists or every intermediate rewrite. Keeping experiment traces separate from ordinary user history can support detailed analysis without making the chat interface more complicated.

Appendix 4
SELECTED SOURCE LISTINGS

The listings appearing next are drawn from the evaluation repository at the specified versions. They illustrate the workings of retrieval, verification, generation, routing and graph logic. The comments and all the docstrings are removed from the listings for space-saving reasons; however, all the executable statements and their order are preserved. The full source with documentation and all auxiliary modules can be found in the cloned repository and at the referenced GitHub site.

pipeline/retrieval.py

```
import re
from typing import Dict, List
from rank_bm25 import BM25Okapi
from langchain_core.documents import Document
from langchain_core.vectorstores import VectorStore
class BM25Index:
    def __init__(self, documents: List[Document]):
        self.documents = documents
        self.tokenized_corpus = [self._tokenize(doc.page_content) for doc in documents]
        self.bm25 = BM25Okapi(self.tokenized_corpus) if self.tokenized_corpus else None
    @staticmethod
    def _tokenize(text: str) -> List[str]:
        return re.findall(r"\w+", text.lower())
    def search(self, query: str, k: int = 4) -> List[Document]:
        if not self.bm25 or not self.documents:
            return []
        tokenized_query = self._tokenize(query)
        if not tokenized_query:
            return []
        scores = self.bm25.get_scores(tokenized_query)
        top_indices = scores.argsort()[::-1][:k]
        return [self.documents[i] for i in top_indices if scores[i] > 0]
def _rrf_rank(
    vector_docs: List[Document],
    keyword_docs: List[Document],
    k: int = 4,
    rrf_k: int = 60,
) -> List[Document]:
    def _doc_key(doc: Document) -> tuple:
        return (
            doc.page_content.strip(),
            doc.metadata.get("source"),
            doc.metadata.get("page"),
        )
```

```
    scores: Dict[tuple, float] = {}
    docs_by_key: Dict[tuple, Document] = {}
    for rank, doc in enumerate(vector_docs, start=1):
        key = _doc_key(doc)
        scores[key] = scores.get(key, 0.0) + 1.0 / (rrf_k + rank)
        docs_by_key[key] = doc
    for rank, doc in enumerate(keyword_docs, start=1):
        key = _doc_key(doc)
        scores[key] = scores.get(key, 0.0) + 1.0 / (rrf_k + rank)
        docs_by_key[key] = doc
    sorted_keys = sorted(scores, key=lambda key: scores[key], reverse=True)
    return [docs_by_key[key] for key in sorted_keys[:k]]
def hybrid_retrieval(
    vectorstore: VectorStore,
    bm25_index: BM25Index,
    query: str,
    k: int = 4,
) -> List[Document]:
    vector_docs = vectorstore.similarity_search(query, k=k)
    keyword_docs = bm25_index.search(query, k=k)
    return _rrf_rank(vector_docs, keyword_docs, k=k)
```

pipeline/verification.py

```
from typing import List
from pydantic import BaseModel, Field
from langchain_core.documents import Document
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import SystemMessage, HumanMessage
class EvidenceVerification(BaseModel):
    enough_evidence: bool = Field(
        description="True ONLY if the retrieved documents contain sufficient factual evidence
    to answer the question; False otherwise."
    )
    reason: str = Field(
        description="Concise justification explaining why the evidence is sufficient or
    insufficient."
    )
def verify_evidence(
    question: str,
    documents: List[Document],
    llm: ChatGoogleGenerativeAI,
) -> EvidenceVerification:
    if not documents:
        return EvidenceVerification(
            enough_evidence=False,
            reason="No documents were retrieved.",
        )
    formatted_context = "\n\n---\n\n".join(
        f"[Source: {doc.metadata.get('source', 'Unknown')}"
        + (f", Page: {doc.metadata['page']}" if "page" in doc.metadata else "")
```

```
        + f"]\n{doc.page_content}"
        for doc in documents
    )
    structured_llm = llm.with_structured_output(EvidenceVerification)
    messages = [
        SystemMessage(
            content=(
                "You are a conservative evidence verifier in a RAG system.\n"
                "Evaluate whether the retrieved context contains sufficient, factual evidence
    "
                "to answer the user's question accurately.\n\n"
                "Guidelines:\n"
                "- Be conservative. If the context merely touches on keywords or is
    incomplete, mark enough_evidence as false.\n"
                "- Only mark enough_evidence as true if the retrieved text contains the
    concrete facts needed to answer the question.\n"
                "- Provide a concise reason for your decision."
            )
        ),
        HumanMessage(
            content=(
                f"Question:\n{question}\n\n"
                f"Retrieved Context:\n{formatted_context}\n\n"
                "Perform evidence verification:"
            )
        ),
    ]
    try:
        result = structured_llm.invoke(messages)
        return result
    except Exception as e:
        return EvidenceVerification(
            enough_evidence=False,
            reason=f"Evidence verification encountered an error: {e}",
        )
```

pipeline/generation.py

```
from typing import List
from pydantic import BaseModel, Field
from langchain_core.documents import Document
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import SystemMessage, HumanMessage
class AnswerOutput(BaseModel):
    answer: str = Field(
        description="The detailed answer based strictly and solely on the provided context."
    )
    confidence: float = Field(
        default=0.9,
        description="An evidence-support confidence score between 0.0 and 1.0 reflecting how
    directly the context supports the answer.",
```

```
    )
    warning: str = Field(
        default="Low risk — answer is directly supported by retrieved documents.",
        description="A concise hallucination risk assessment string.",
    )
def format_sources(documents: List[Document]) -> List[str]:
    sources: List[str] = []
    seen = set()
    for doc in documents:
        source_name = doc.metadata.get("source", "Unknown")
        page = doc.metadata.get("page")
        if page is not None:
            citation = f"{source_name}, page {page}"
        else:
            citation = source_name
        if citation not in seen:
            seen.add(citation)
            sources.append(citation)
    return sources
def generate_answer(
    question: str,
    documents: List[Document],
    llm: ChatGoogleGenerativeAI,
) -> AnswerOutput:
    formatted_context = "\n\n---\n\n".join(
        f"[Source: {doc.metadata.get('source', 'Unknown')}"
        + (f", Page: {doc.metadata['page']}" if "page" in doc.metadata else "")
        + f"]\n{doc.page_content}"
        for doc in documents
    )
    structured_llm = llm.with_structured_output(AnswerOutput)
    messages = [
        SystemMessage(
            content=(
                "You are a precise RAG assistant. Answer the question using ONLY the provided
    context.\n\n"
                "Rules:\n"
                "1. If the context does not explicitly support a claim, do NOT make the
    claim.\n"
                "2. Do NOT use outside or prior knowledge.\n"
                "3. Provide a confidence score between 0.0 and 1.0 reflecting the factual
    evidence strength.\n"
                "4. In the warning field, always provide a concise risk assessment statement "
                "(e.g., 'Low risk — answer is directly supported by retrieved documents.').
    Never leave it empty."
            )
        ),
        HumanMessage(
            content=(
                f"Question:\n{question}\n\n"
```

```
            f"Retrieved Context:\n{formatted_context}\n\n"
            "Generate the grounded response:"
        )
    ),
]
try:
    result = structured_llm.invoke(messages)
    result.confidence = max(0.0, min(1.0, float(result.confidence)))
    if not result.warning or not result.warning.strip():
        result.warning = "Low risk — answer is directly supported by retrieved documents."
    return result
except Exception as e:
    return AnswerOutput(
        answer=f"Error generating answer: {e}",
        confidence=0.0,
        warning="High risk — generation encountered an error.",
    )
```

pipeline/query.py

```
from typing import List, Optional
from langchain_core.output_parsers import StrOutputParser
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import BaseMessage, SystemMessage, HumanMessage
_str_parser = StrOutputParser()
def _parse_llm_response(response) -> str:
    return _str_parser.invoke(response).strip()
def understand_query(
    question: str,
    llm: ChatGoogleGenerativeAI,
    chat_history: Optional[List[BaseMessage]] = None,
) -> str:
    history_context = ""
    if chat_history and len(chat_history) > 1:
        recent = chat_history[-5:-1]
        if recent:
            history_lines = []
            for m in recent:
                role = "User" if isinstance(m, HumanMessage) else "Assistant"
                history_lines.append(f"{role}: {m.content}")
            history_context = "Recent Conversation Context:\n" + "\n".join(history_lines) +
    "\n\n"
    messages = [
        SystemMessage(
            content=(
                "You are a search query optimizer for a RAG system.\n"
                "Transform the user's input question into a concise, keyword-focused search
    query.\n"
                "Resolve any coreferences or pronouns (e.g., 'it', 'they', 'that') using the
    recent conversation context if relevant.\n"
                "Rules:\n"
```

```
            "- Do NOT answer the question.\n"
            "- Do NOT include explanations, markdown, or punctuation wrappers.\n"
            "- Output ONLY the plain search query string."
        )
    ),
    HumanMessage(content=f"{history_context}Current Question: {question}"),
  ]
  response = llm.invoke(messages)
  query = _parse_llm_response(response).strip("").strip("'")
  return query if query else question
def improve_query(
  question: str,
  previous_query: str,
  reason: str,
  llm: ChatGoogleGenerativeAI,
) -> str:
  messages = [
    SystemMessage(
      content=(
        "You are an expert query refinement optimizer for a RAG system.\n"
        "The previous search query failed to retrieve sufficient evidence to answer
  the question.\n"
        "Refine the query by:\n"
        "1. Preserving the user's core intent.\n"
        "2. Adding technical synonyms, related terms, acronyms, or specific
  concepts.\n"
        "3. Removing generic or conversational filler words.\n"
        "Rules:\n"
        "- Do NOT answer the question.\n"
        "- Do NOT include explanations or punctuation wrappers.\n"
        "- Output ONLY the refined search query string."
      )
    ),
    HumanMessage(
      content=(
        f"Original Question: {question}\n"
        f"Previous Query: {previous_query}\n"
        f"Verification Failure Reason: {reason}\n"
        "Generate improved search query:"
      )
    ),
  ]
  response = llm.invoke(messages)
  refined = _parse_llm_response(response).strip("").strip("'")
  return refined if refined else previous_query
def check_retrieval_needed(
  question: str,
  llm: ChatGoogleGenerativeAI,
  chat_history: Optional[List[BaseMessage]] = None,
) -> bool:
```

```
cleaned = question.strip().lower().rstrip("!?.")
common_greetings = {
    "hi", "hello", "hey", "hey there", "greetings", "good morning",
    "good afternoon", "good evening", "how are you", "how are you doing",
    "how's it going", "who are you", "what can you do", "what are you",
    "help", "thanks", "thank you", "bye", "goodbye", "see you", "ok",
    "okay", "cool", "nice", "great", "welcome",
}
if cleaned in common_greetings:
    return False
words = cleaned.split()
if len(words) == 1 and words[0] in {"hi", "hello", "hey", "hola", "yo", "sup", "thanks",
"thx"}:
    return False
conversational_phrases = {
    "what is my name", "what is me name", "what's my name", "who am i",
    "do you know my name", "do you remember my name", "tell me my name",
    "what did i say", "what did i ask", "what was my last question",
    "my name is", "my name i", "i am", "call me",
}
if any(cleaned == phrase or cleaned.startswith(phrase + " ") for phrase in
conversational_phrases):
    return False
history_context = ""
if chat_history and len(chat_history) > 1:
    recent = chat_history[-5:-1]
    if recent:
        history_lines = []
        for m in recent:
            role = "User" if isinstance(m, HumanMessage) else "Assistant"
            history_lines.append(f"{role}: {m.content}")
        history_context = "Recent Conversation History:\n" + "\n".join(history_lines) +
"\n\n"
messages = [
    SystemMessage(
        content=(
            "You are an intent router for a Retrieval-Augmented Generation (RAG)
assistant.\n"
            "Analyze the user's latest input and decide whether answering it requires
document retrieval from a knowledge base, or if it is purely conversational or refers to the
ongoing conversation/user identity.\n\n"
            "Criteria:\n"
            "- Output 'NO' if the input is a greeting, polite pleasantry, small talk,
expression of gratitude, farewell, asking what you can do, or asking about the user/chat
history (e.g., 'what is my name', 'who am I', 'what did I say earlier').\n"
            "- Output 'YES' if the input asks for specific facts, explanations, data,
policies, summaries, or questions from documents.\n\n"
            "Rules:\n"
            "- Output ONLY 'YES' or 'NO' with no extra words or punctuation."
        )
```

```
        ),
        HumanMessage(content=f"{history_context}User input: {question}"),
    ]
    try:
        response = llm.invoke(messages)
        decision = _parse_llm_response(response).upper()
        return "NO" not in decision
    except Exception:
        return True
```

pipeline/graph.py

```
import operator
from typing import TypedDict, List, Annotated, Sequence, Optional
from langgraph.graph import StateGraph, START, END
from langchain_core.documents import Document
from langchain_core.vectorstores import VectorStore
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import BaseMessage, SystemMessage, HumanMessage,
AIMessage
from pipeline.retrieval import BM25Index, hybrid_retrieval
from langchain_core.output_parsers import StrOutputParser
from pipeline.query import understand_query, improve_query, check_retrieval_needed
from pipeline.verification import verify_evidence
from pipeline.generation import generate_answer, format_sources
from pipeline.web_search import tavily_search
import config
class RAGState(TypedDict, total=False):
    messages: Annotated[List[BaseMessage], operator.add]
    question: str
    search_query: str
    needs_retrieval: bool
    documents: List[Document]
    enough_evidence: bool
    verification_reason: str
    answer: str
    confidence: float
    warning: str
    sources: List[str]
    iterations: int
def build_rag_graph(
    vectorstore: VectorStore,
    bm25_index: BM25Index,
    llm: ChatGoogleGenerativeAI,
    k: int = config.RETRIEVAL_K,
    max_iterations: int = config.MAX_ITERATIONS,
):
    def check_retrieval_needed_node(state: RAGState) -> dict:
        needs_retrieval = check_retrieval_needed(
            state["question"],
            llm,
```

```
        chat_history=state.get("messages", []),
    )
    return {"needs_retrieval": needs_retrieval}
def direct_chat_node(state: RAGState) -> dict:
    history = state.get("messages", [])
    prompt_messages = [
        SystemMessage(
            content=(
                "You are a friendly, helpful AI RAG assistant with access to a document
knowledge base.\n"
                "Respond warmly, helpfully, and conversationally.\n"
                "You have access to the conversation history. If the user asks about
details mentioned earlier in the chat (such as their name, previous questions, or personal
remarks), answer accurately and directly using that context.\n"
                "Keep conversational responses natural and concise. Remind them they can
also ask questions about the loaded documents."
            )
        ),
    ]
    if history:
        prompt_messages.extend(history)
    else:
        prompt_messages.append(HumanMessage(content=state["question"]))
    try:
        response = llm.invoke(prompt_messages)
        answer = StrOutputParser().invoke(response).strip()
    except Exception:
        answer = "Hello! How can I help you today? Feel free to ask me anything based on
your documents."
    return {
        "messages": [AIMessage(content=answer)],
        "answer": answer,
        "confidence": 1.0,
        "warning": "Conversational response — no document retrieval needed.",
        "sources": [],
        "enough_evidence": True,
    }
def decide_retrieval_route(state: RAGState) -> str:
    if state.get("needs_retrieval", True):
        return "understand_query"
    return "direct_chat"
def understand_query_node(state: RAGState) -> dict:
    query = understand_query(
        state["question"],
        llm,
        chat_history=state.get("messages", []),
    )
    return {
        "search_query": query,
        "iterations": state.get("iterations", 0),
```

```
    }
def retrieve_node(state: RAGState) -> dict:
    query = state.get("search_query", state["question"])
    docs = hybrid_retrieval(vectorstore, bm25_index, query, k=k)
    current_iter = state.get("iterations", 0) + 1
    return {
        "documents": docs,
        "iterations": current_iter,
    }
def verify_evidence_node(state: RAGState) -> dict:
    docs = state.get("documents", [])
    verification = verify_evidence(state["question"], docs, llm)
    return {
        "enough_evidence": verification.enough_evidence,
        "verification_reason": verification.reason,
    }
def improve_query_node(state: RAGState) -> dict:
    refined_query = improve_query(
        question=state["question"],
        previous_query=state.get("search_query", state["question"]),
        reason=state.get("verification_reason", "Insufficient supporting evidence"),
        llm=llm,
    )
    return {
        "search_query": refined_query,
    }
def web_search_node(state: RAGState) -> dict:
    query = state.get("search_query", state["question"])
    web_docs = tavily_search(query)
    return {
        "documents": web_docs,
    }
def generate_answer_node(state: RAGState) -> dict:
    docs = state.get("documents", [])
    output = generate_answer(state["question"], docs, llm)
    sources = format_sources(docs)
    warning = output.warning
    if any(doc.metadata.get("is_web") for doc in docs):
        warning = "Notice — answer was retrieved via Tavily Web Search fallback because
local documents lacked sufficient coverage."
    return {
        "messages": [AIMessage(content=output.answer)],
        "answer": output.answer,
        "confidence": output.confidence,
        "warning": warning,
        "sources": sources,
    }
def handle_insufficient_evidence_node(state: RAGState) -> dict:
    docs = state.get("documents", [])
    sources = format_sources(docs) if docs else []
```

```
    ans = "I could not find sufficient evidence in the provided documents or web search to
 answer this question reliably."
    return {
        "messages": [AIMessage(content=ans)],
        "answer": ans,
        "confidence": 0.1,
        "warning": "High risk — insufficient supporting evidence was found after maximum
retrieval attempts and web search.",
        "sources": sources,
    }
def decide_next_step(state: RAGState) -> str:
    if state.get("enough_evidence", False):
        return "generate_answer"
    if state.get("iterations", 0) < max_iterations:
        return "improve_query"
    return "web_search"
def decide_after_web_search(state: RAGState) -> str:
    docs = state.get("documents", [])
    if docs:
        return "generate_answer"
    return "handle_insufficient_evidence"
workflow = StateGraph(RAGState)
workflow.add_node("check_retrieval_needed", check_retrieval_needed_node)
workflow.add_node("direct_chat", direct_chat_node)
workflow.add_node("understand_query", understand_query_node)
workflow.add_node("retrieve", retrieve_node)
workflow.add_node("verify_evidence", verify_evidence_node)
workflow.add_node("improve_query", improve_query_node)
workflow.add_node("web_search", web_search_node)
workflow.add_node("generate_answer", generate_answer_node)
workflow.add_node("handle_insufficient_evidence", handle_insufficient_evidence_node)
workflow.add_edge(START, "check_retrieval_needed")
workflow.add_conditional_edges(
    "check_retrieval_needed",
    decide_retrieval_route,
    {
        "understand_query": "understand_query",
        "direct_chat": "direct_chat",
    },
)
workflow.add_edge("direct_chat", END)
workflow.add_edge("understand_query", "retrieve")
workflow.add_edge("retrieve", "verify_evidence")
workflow.add_conditional_edges(
    "verify_evidence",
    decide_next_step,
    {
        "generate_answer": "generate_answer",
        "improve_query": "improve_query",
        "web_search": "web_search",
```

```
    },
)
workflow.add_edge("improve_query", "retrieve")
workflow.add_conditional_edges(
    "web_search",
    decide_after_web_search,
    {
        "generate_answer": "generate_answer",
        "handle_insufficient_evidence": "handle_insufficient_evidence",
    },
)
workflow.add_edge("generate_answer", END)
workflow.add_edge("handle_insufficient_evidence", END)
return workflow.compile()
```